

VERİ YAPILARI DERS ANLATIM SÜRECİNDE GEREKECEK TEMEL VE ÖN BİLGİLER

Dr. Öğr. Üyesi Fatma AKALIN

1. Donanım (Hardware) Nedir?

Donanım, bilgisayarın fiziksel bileşenlerinden oluşan sistemdir. İşlemci (CPU), anakart, bellek (RAM), depolama birimleri (SSD/HDD), giriş/çıkış birimleri (klavye, fare, ekran) gibi bileşenleri içerir.

2. Bellek (Memory) Nedir?

Bellek, bilgisayarın veri saklama ve işleme sürecinde kullandığı alanları ifade eder. İki ana kategoriye ayrılır:

Birincil Bellek (Primary Memory): RAM, önbellek (cache) ve işlemci kayıtları (registers) gibi geçici ve hızlı erişimli bellek türlerini içerir. İşlemcinin doğrudan erişebildiği bellektir.

İkincil Bellek (Secondary Storage): Sabit diskler (HDD, SSD), flash bellekler, optik diskler (CD, DVD) gibi kalıcı veri depolama birimleridir.

3. Donanım ve Bellek Arasındaki İlişki

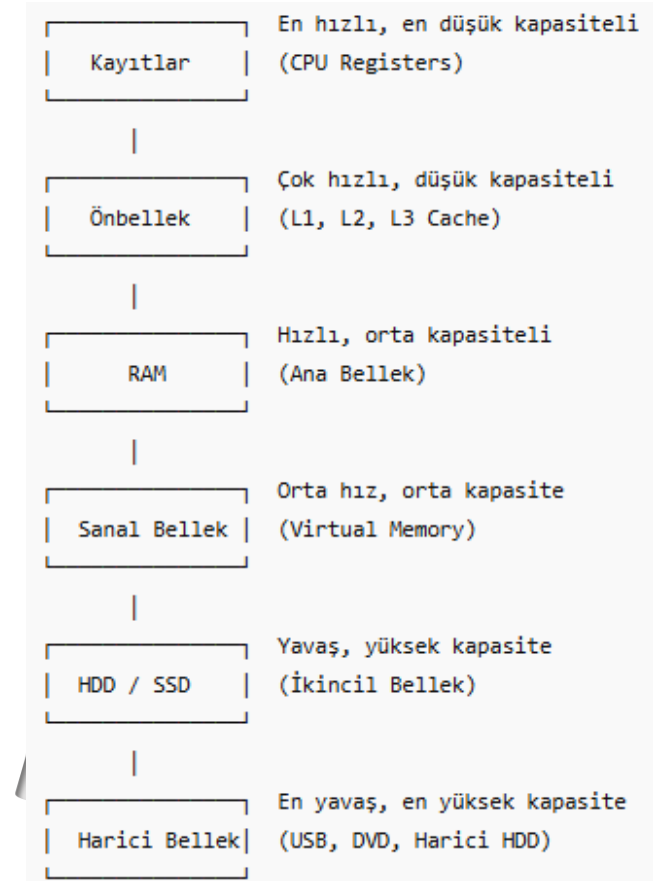
İşlemci (CPU) ile Bellek: İşlemci, komutları çalıştırmak için belleğe erişir. Önce önbellekten (cache), sonra RAM'den ve en sonunda sabit diskten veri alır.

Önbellek (Cache) ile RAM: İşlemci, daha hızlı işlem yapabilmek için sık kullanılan verileri önbelleğe alır. Eğer önbellekte bulunmazsa, RAM'e yönelir.

RAM ile Depolama (HDD/SSD): RAM, verileri hızlı bir şekilde saklar ve işlemciye iletir. Uzun süreli saklama gerektiğinde veriler sabit diske yazılır.

Bu ilişkiler sayesinde bilgisayar sistemleri verimli çalışır. Hızlı erişim için **önbellek > RAM > HDD/SSD** sıralaması kullanılır.

Bellek Türü	Erişim Hızı	Kapasite	Kalıcılık	Örnekler
İşlemci Kayıtları (Registers)	En hızlı	En düşük	Geçici	CPU kayıtları
Önbellek Bellek (Cache)	Çok hızlı	Düşük	Geçici	L1, L2, L3 önbellek
Ana Bellek (RAM)	Hızlı	Orta	Geçici	DDR4, DDR5 RAM
Sanal Bellek (Virtual Memory)	Orta	Orta	Geçici	Sayfalama dosyaları (Windows Pagefile, Linux Swap)
İkincil Bellek (HDD/SSD)	Yavaş	Yüksek	Kalıcı	Sabit disk, SSD
Harici Bellek (External Storage)	Daha yavaş	Yüksek	Kalıcı	USB bellek, harici HDD, DVD



Register (Kayıt):

•**Tanım:** Register, işlemcinin (CPU) içinde bulunan çok hızlı ve küçük veri depolama birimleridir. Bu birimler, işlemci tarafından kullanılan veriyi hızlı bir şekilde tutmak için tasarlanmıştır.

•**Kapasite:** Kayıtlar, genellikle birkaç bitten oluşur ve çok sınırlı kapasiteye sahiptir (örneğin, 32-bit ya da 64-bit).

•**Erişim Hızı:** En hızlı veri erişimi sağlayan depolama birimidir çünkü işlemcinin doğrudan erişebildiği bir alan içinde bulunurlar.

•**Kullanım Amacı:** Hesaplamalar sırasında, örneğin, işlemci üzerinde yapılan aritmetik ve mantıksal işlemlerde geçici verileri depolamak için kullanılır. Bu, işlemcinin hızlı çalışmasına yardımcı olur.

Cache (Önbellek):

•**Tanım:** Cache, veri ve komutların geçici olarak saklandığı daha hızlı erişilebilen bir bellek alanıdır. Cache, ana belleğe (RAM) göre çok daha hızlıdır ve işlemcinin işlem hızını artırmak için kullanılır.

•**Kapasite:** Cache, register'dan çok daha büyük olabilir, genellikle kilobaytlar ile megabaytlar arasında değişir. CPU'da birkaç seviyede cache olabilir (L1, L2, L3).

•**Erişim Hızı:** Cache bellek, işlemciye daha yakın yer alır ve RAM'e göre çok daha hızlı erişim sağlar, ancak register kadar hızlı değildir.

•**Kullanım Amacı:** İşlemcinin daha önce kullandığı veya sıkça erişeceği verilere hızlı erişim sağlamaktır. Cache, daha uzun sürede erişilebilen verilere (ana bellek gibi) yapılan erişimi hızlandırmak amacıyla kullanılır.

Bu şema, hızın yukarıdan aşağıya doğru azaldığını, ancak kapasitenin arttığını gösterir.

RAM (Rastgele Eriřimli Bellek - Random Access Memory):

•**Tanım:** RAM, bilgisayarın iřlem yapabilmesi iin geici olarak veri saklayan bellek trdr. Bilgisayar aıldığında iřletim sistemi ve uygulamalar RAM'e yklenir. RAM, uucu (volatile) bir bellek trdr, yani bilgisayar kapandığında iindeki veriler kaybolur.

•**Kapasite ve Hız:** RAM genellikle yksek hızda veri eriřimi saėlar ve bilgisayarın iřlemciye veri iletmesi iin kullanılan geici depolama alanıdır. Genellikle gigabayt (GB) cinsinden lr.

•**Kullanım Amacı:** Uygulamalar ve iřletim sistemi iin aktif alıřma alanı saėlar. CPU'nun hızlı eriřebilmesi iin veriler burada depolanır.

Sanal Bellek:

•**Tanım:** Sanal bellek, RAM'in kapasitesinin ařılması durumunda iřletim sistemi tarafından kullanılan bir bellek ynetim tekniėidir. RAM dolarsa, disk zerinde belirli bir alan (genellikle sabit disk ya da SSD) sanal bellek olarak kullanılmaya bařlanır. Bu iřlem, gerek RAM gibi alıřıyormuř gibi grnse de, aslında daha yavař bir veri eriřimi saėlar.

•**Kapasite ve Hız:** Sanal bellek, fiziksel RAM'in kapasitesine ek olarak disk alanını kullanarak daha fazla uygulama alıřtırabilmeyi saėlar. Ancak, RAM ile kıyaslandığında eriřim hızları ok daha dřktr, nk diskler (zellikle sabit diskler) ok daha yavař alıřır.

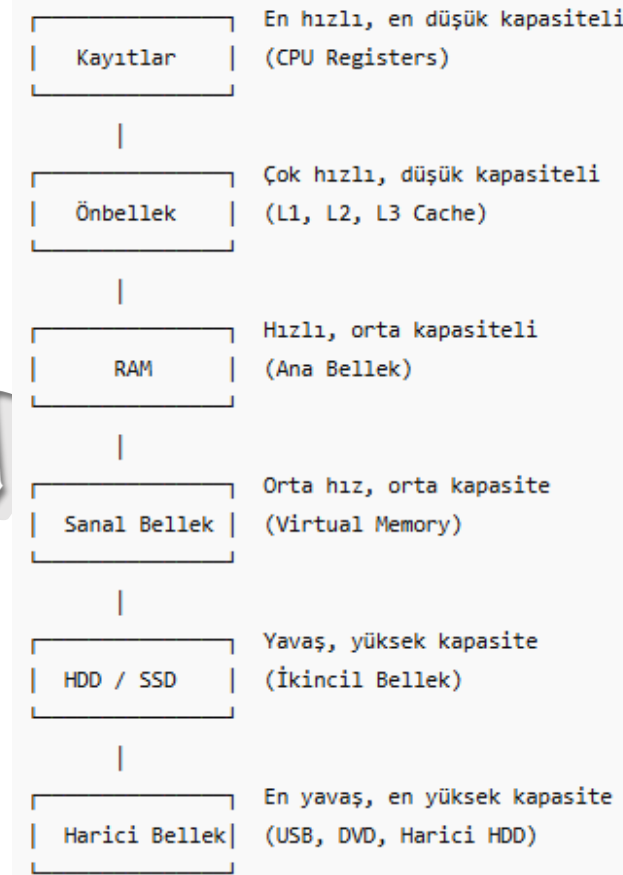
•**Kullanım Amacı:** RAM'in dolduėunda, kullanılan verilerin bir kısmı sabit diske (veya SSD'ye) kaydedilerek RAM'in serbest kalmasını saėlar. Bu, sistemin daha fazla uygulama veya iřlemi desteklemesine olanak tanır, ancak hızda bir dřř yařlanır.

İkincil Bellek:

•**Tanım:** İkincil bellek, kalıcı veri depolama için kullanılan bellek türüdür. RAM'den farklı olarak, veriler burada kalıcıdır ve bilgisayar kapansa bile silinmez. İkincil bellek, genellikle sabit diskler (HDD), katı hal sürücüsü (SSD) ve optik diskler (CD, DVD) gibi cihazlarda bulunur.

•**Kapasite ve Hız:** İkincil bellek, genellikle büyük kapasiteye sahiptir (terabaytlarca veriye kadar) ancak veri erişim hızı RAM'e kıyasla çok daha düşüktür. SSD'ler, HDD'lere göre daha hızlıdır, ancak hala RAM ile kıyaslanacak kadar hızlı değildir.

•**Kullanım Amacı:** Verilerin uzun süreli saklanması ve işletim sistemi, uygulamalar, oyunlar, medya dosyaları ve diğer büyük veri dosyalarının depolanması için kullanılır.



Bu şema, hızın yukarıdan aşağıya doğru azaldığını, ancak kapasitenin arttığını gösterir.

Harici Bellek:

•**Tanım:** Harici bellek, bilgisayarın dışında bulunan ve genellikle USB bellekler, harici sabit diskler, SSD'ler, CD/DVD diskler ve SD kartlar gibi taşınabilir depolama birimleriyle sağlanan bellek türüdür. Bu bellekler, veri taşıma ve yedekleme için kullanılır.

•**Kapasite ve Hız:** Harici bellekler, kapasite açısından çok büyük olabilir (terabaytlarca veri), ancak hızları bağlı oldukları arabirimlere göre değişir. Örneğin, USB 2.0 ve USB 3.0 arasındaki hız farkı oldukça büyüktür, aynı şekilde bir harici SSD ile bir harici HDD'nin hızları da farklıdır.

•**Kullanım Amacı:** Verilerin taşınması, yedeklenmesi, depolanması veya paylaşılması amacıyla kullanılır. Harici bellekler, verileri ana bilgisayardan bağımsız bir şekilde saklamak ve taşıma imkanı sunar.

Donanım ve yazılımın ilişkisi şu şekilde
özetlenebilir

1-Yazılımlar ve Bellek Yükleme: Yazılımlar, çalıştırılmak üzere **diskten** (HDD/SSD) **RAM**'e yüklenir. Çünkü işlemci, doğrudan diskten veri alamaz; bu yüzden yazılımın ve çalışması için gereken verilerin önce RAM'e yüklenmesi gerekir. RAM, işlemcinin hızlı erişebileceği geçici bir depolama alanıdır.

2-İşlemci ve Bellek Erişimi: İşlemci, RAM'den komutları alır ve işler. Ancak bu işlemde verinin en hızlı şekilde erişilmesi için önce **cache** kullanılır. Eğer işlemci, ihtiyaç duyduğu veri cache'de bulunuyorsa, çok hızlı erişim sağlanır. Aksi takdirde, işlemci veriye erişmek için daha yavaş olan **RAM**'e başvurur.

3-İşletim Sistemi ve Bellek Yönetimi: İşletim sistemi, yazılımların çalışabilmesi için belleği yönetir. Bu yönetimde **sanal bellek** kullanımı da kullanılır. Sanal bellek, RAM'in kapasitesinin yetersiz olduğu durumlarda devreye girer ve HDD/SSD üzerinde bir alan tahsis edilerek RAM ile birlikte çalışır.

5-Yedekleme Yazılımları: Yedekleme yazılımları, önemli verileri kaybolma riskine karşı yedekleme birimlerine aktarır. Bu yedekleme işlemi genellikle **harici bellek** birimlerine (USB bellek, harici sabit disk, bulut vb.) yapılır. Bu işlemler, verilerin dışarıya taşınarak güvence altına alınması için önemlidir.

4-Verilerin Kalıcı Saklanması (HDD/SSD): İşlemcinin işlediği veriler ve uygulama, RAM'de geçici olarak saklanır, ancak veriler kalıcı olarak saklanmak istendiğinde **HDD** veya **SSD** gibi **ikincil bellek** birimleri kullanılır. Bu bellekler, bilgisayarın kapanıp açılması durumunda bile verilerin kaybolmamasını sağlar. Yani, yazılımlar ve veriler son haliyle HDD/SSD'ye yazılır.

Özet:

- **Register'lar** işlemcinin içindeki en hızlı bellek alanlarıdır ve sadece işlemcinin işlediği geçici verileri tutar.
 - **Cache** ise işlemcinin hızlı erişebileceği küçük ve hızlı bir bellek alanıdır. Veriler cache'e taşındığında, işlemci çok daha hızlı çalışabilir.
 - **RAM**, yazılımlar ve verilerin çalışabilmesi için kullanılan ana bellek türüdür ve işlemcinin erişebileceği verileri sağlar.
 - **Sanal bellek**, RAM'in kapasitesini aşan durumlarda devreye girer ve HDD/SSD üzerinde verileri geçici olarak saklar.
 - **HDD/SSD** ve **harici bellek** gibi ikincil bellek birimleri, verilerin kalıcı olarak saklandığı ve yedeklendiği alanlardır.
- Her bir bellek türü, farklı ihtiyaçlar için kullanılır ve bilgisayarın genel performansını iyileştirmeye yardımcı olur.

BİLGİSAYAR ORGANİZASYONU

Mantıksal Birim	Açıklama
Hafıza Birimi	Hafıza birimi, giriş biriminden gelen verilerin saklandığı ve sahip olduğu verilere hızlı erişim imkânı tanıyan düşük kapasiteli "depo" birimidir. İşlenmesi istenen verinin hızlı bir şekilde erişilmesine imkân verir. Hafıza birimi, çıkış birimi tarafından çıkış cihazlarına gönderilmeyi bekleyen işlenmiş verileri de muhafaza edebilmektedir. Hafıza birimindeki veriler geçicidir. Bilgisayarın elektriği kesildiğinde veriler kaybolur. Hafıza birimi genelde hafıza veya birincil hafıza olarak adlandırılır. Masaüstü ve dizüstü bilgisayarlarının ana hafızaları 16 GB kadar büyük olabilmektedir. Bir gigabayt yaklaşık olarak bir milyar bayttır.
Aritmetik ve Mantıksal Birim (ALU)	Bu birimin görevi toplama, çıkarma, çarpma ve bölme gibi hesaplamaları gerçekleştirmektir. Ayrıca, hafıza birimindeki iki verinin eşit olup olmadığına karar verme gibi mantıksal kararlarda bu birimde verilmektedir. ALU genelde CPU'nun bir parçası olarak geliştirilmektedir.
Merkezi İşlem Birimi (CPU)	Bu yönetici birim diğer birimlerin operasyonlarını koordine etmekle görevlidir. CPU, giriş birimine verilerin ne zaman hafıza birimine yükleneceğini, ALU birimine hafıza birimindeki verilerin ne zaman işleneceğini, çıkış birimine hafıza biriminden alınan verilerin ne zaman çıkış cihazlarına gönderileceğini söyler. Günümüzdeki çoğu bilgisayar birden fazla işlemciye sahiptir. Bu sayede birden fazla işlem aynı anda yapılabilir. Çok çekirdekli işlemciler, bir silikon çip içerisinde birden fazla işlemciye sahiptir. Günümüz masaüstü bilgisayarları saniye milyarlarca komutu gerçekleştirebilmektedir.
İkincil Saklama Birimi	Bu birim uzun ömürlü ve yüksek kapasiteli depo birimidir. Diğer birimler tarafından aktif olarak kullanılmayan verilerin ve programların, ihtiyaç duyulana kadar saklanması için ikincil saklama birimine yerleştirilir. İkinci saklama birimindeki veriler kalıcıdır. Bilgisayarın enerjisi kesildiğinde dahi içindeki veriler kaybolmaz. İkincil saklama birimindeki verilere erişmek birincil hafıza birimine erişmekten çok daha yavaştır. Fakat hafıza birimi başına düşen fiyat birincil hafızadan çok daha düşüktür. CD, DVD ve flaş sürücüler ikincil saklama birimi cihazlarına örnek olarak verilebilir. Günümüz masaüstü ve dizüstü bilgisayarlarının sabit diskleri 2 terabayta kadar veri saklayabilmektedir.

Bilgisayarlar, farklı birimlerden oluşan karmaşık sistemlerdir. Her birim, belirli bir işlevi yerine getirir ve bu birbirleriyle etkileşim halinde çalışır.

Hafıza birimi, ALU, CPU ve ikincil saklama birimi; bilgisayarın temel yapı taşlarıdır ve her biri belirli işlevlere sahiptir. Şimdi, her birinin ne olduğunu ve bellekle nasıl ilişkili olduklarını açıklayayım:

Fatma AKALIN

1. Hafıza Birimi (Memory Unit):

Hafıza birimi, verilerin ve programların geçici olarak saklandığı yerdir. Bilgisayarın işlemciye veri sağlamak için kullandığı temel bellek alanıdır. **RAM (Rastgele Erişimli Bellek)** ve **ROM (Okunabilir Sadece Bellek)** gibi belleklere örnek verilebilir. **İlişkisi:** Hafıza birimi, CPU'nun ve diğer donanımların hızlı erişebilmesi için kritik öneme sahiptir. RAM, geçici veri depolama için kullanılırken, ROM daha kalıcı, sistemsel verileri içerir.

4. İkincil Saklama Birimi (Secondary Storage):

İkincil saklama birimi, verilerin kalıcı olarak saklandığı, genellikle daha büyük ve daha yavaş depolama alanıdır. Bu, bilgisayarın "sabit disk" ya da **SSD** (Kati Hal Sürücüsü) gibi cihazları içerir. İkincil saklama birimi, bilgisayarın kapalı olduğu zamanlarda bile verilerin kaybolmaması için kullanılır.

•**İlişkisi:** İkincil saklama birimi, RAM'den farklı olarak kalıcı veri depolama alanı sağlar. İşletim sistemi, uygulama programları ve kullanıcı verileri genellikle ikincil saklama birimlerinde saklanır. CPU ve RAM, geçici işlem yaparken, ikincil saklama birimi verileri uzun süreli depolamak için kullanılır.

2. ALU (Aritmetik ve Mantıksal Birim - Arithmetic and Logic Unit):

ALU, işlemcinin veri işleme birimidir. Aritmetik işlemler (toplama, çıkarma, çarpma, bölme) ve mantıksal işlemler (karşılaştırmalar, AND, OR gibi mantıksal işlemler) burada yapılır.

•**İlişkisi:** ALU, işlem yapmak için veriye ihtiyaç duyar. Bu veriler, CPU'ya bağlı bellek birimlerinden alınır (genellikle RAM'den). Yani, ALU'nun işlevlerini yerine getirebilmesi için işlemci belleğinden veri alması gerekir.

3. CPU (Merkezi İşlem Birimi - Central Processing Unit):

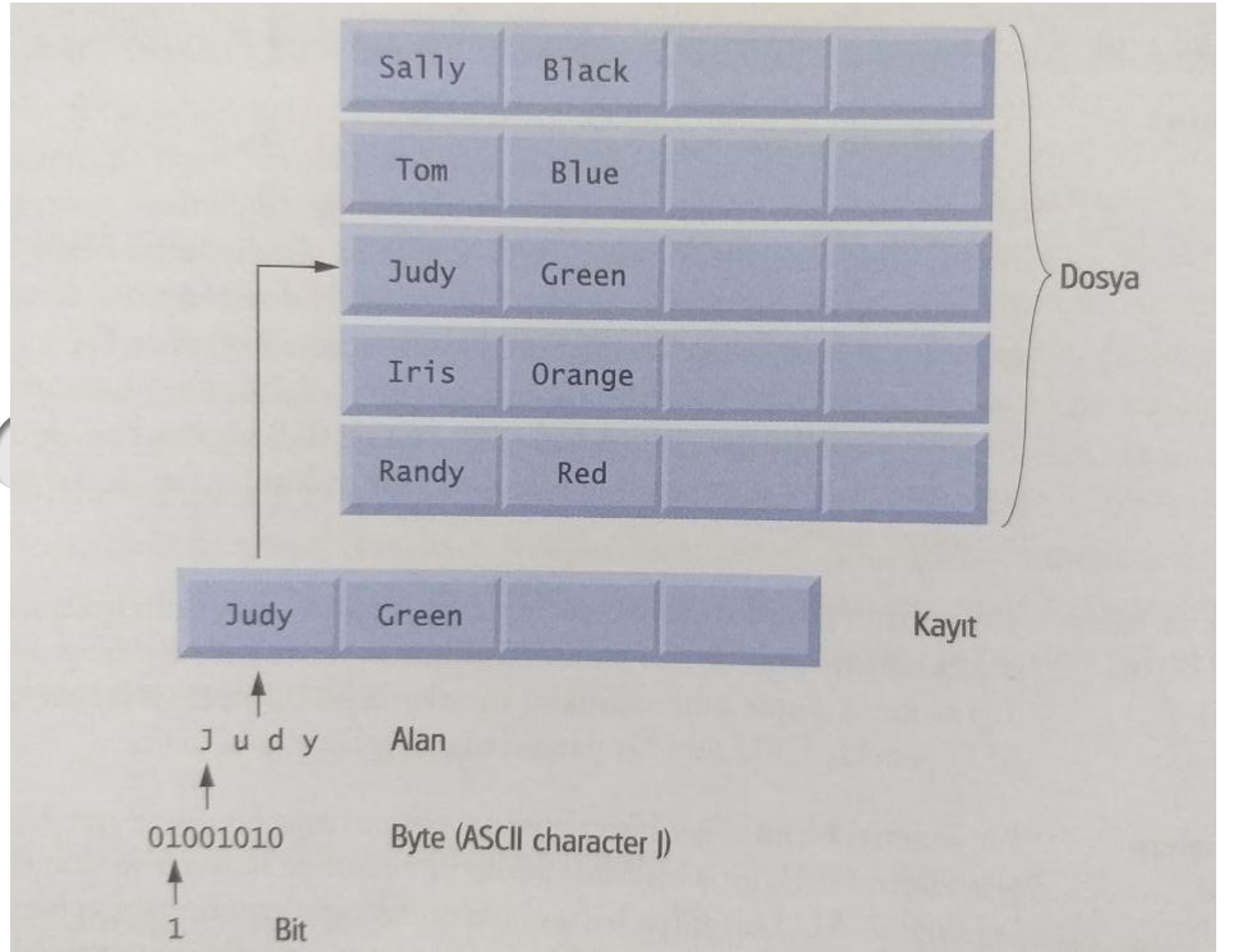
CPU, bilgisayarın "beyni" olarak kabul edilir ve programların komutlarını işler. CPU'nun içinde birçok alt birim bulunur, bunlar arasında **ALU (Aritmetik ve Mantıksal Birim)** ve **Kontrol Birimi** yer alır. CPU'nun temel işlevi, bellekteki verileri alıp bu veriler üzerinde işlem yapmaktır.

•**İlişkisi:** CPU, bellek ile doğrudan etkileşime girer. Veriler önce bellekte saklanır (RAM), ardından CPU bu veriler üzerinde işlem yapar. Bu işlem sonrasında sonuçlar yine bellek birimlerine yazılabilir veya başka işlemler için kullanılmak üzere tekrar alınabilir.

Dr. Öğr. Üyesi Fatma AKALIN

VERİ HİYERARŞİSİ

Bilgisayar tarafından işlenen veriler arka planda gittikçe büyüyen ve derinleşen bir veri hiyerarşisine dönüşmektedir. Yanda ki şekilde veri hiyerarşisinin bir bölümü gösterilmektedir.



Veri Hiyerarşisi Seviyeleri

Dr.

Seviye	Tanım
Bit	Bilgisayar sistemlerinde en küçük veri birimidir. 0 veya 1 değerini alabilir.
Karakter	Bir veya birkaç bitin birleşmesiyle oluşur. Harf, rakam veya sembol olabilir.
Alan	Bir karakter veya karakter grubundan oluşan anlamlı veri birimidir. Örneğin, "Ad" veya "Soyad" bir alan olabilir.
Kayıt	Birbiriyle ilişkili alanların birleşiminden oluşur. Örneğin, bir kişinin adı, soyadı, doğum tarihi gibi bilgileri içeren bir kayıt olabilir.
Dosya	Aynı türden kayıtların bir araya gelmesiyle oluşur. Örneğin, bir çalışan listesi dosyası, çalışanlara ait tüm kayıtları içerebilir.
Veritabanı	Birden fazla dosyanın organize edilerek saklandığı büyük veri yapısıdır. Farklı dosyalar arasındaki ilişkileri yönetir.

W

Makine Dilleri, Sembolik Diller ve Yüksek Seviyeli Diller

Bilgisayarların anlayabileceği diller, zaman içinde teknolojinin gelişmesiyle önemli değişiklikler göstermiştir. İlk zamanlardan günümüze kadar kullanılan dillerin evrimi, bilgisayarların daha hızlı, daha esnek ve daha kullanıcı dostu hale gelmesini sağlamıştır. Bu evrimi üç ana kategoriye ayırabiliriz: **Makine Dilleri**, **Sembolik Diller** ve **Yüksek Seviyeli Diller**. Bu dillerin tarihsel gelişimi, her birinin bilgisayar sistemleri ve yazılım geliştirme üzerindeki etkilerini anlamamıza yardımcı olur.

Makine Dilleri (Machine Languages):

Bilgisayarın anlayabileceği ilk dil, **makine dili**di. Bu dil, bilgisayarın **işlemci** (CPU) tarafından doğrudan anlaşılan ikili (binary) formattır. Yani, her komut ve veri, yalnızca **0** ve **1**'lerden oluşur. Bu, bilgisayarın donanımıyla tamamen uyumlu bir dil olduğu için, bilgisayarın çalıştığı en temel seviyedeki dil olarak kabul edilir.

Özellikleri:

Düşük seviyeli bir dildir ve sadece bilgisayarın işlemcisi tarafından anlaşılır.

Her komut, belirli bir işlemci modeline özeldir ve işlemciye doğrudan komut verir.

İnsanlar için oldukça zor anlaşılır ve yazılması zaman alıcıdır.

Tarihsel Gelişim:

İlk bilgisayarlar sadece makine dilini kullanabiliyorlardı. Programcılar, her bir işlem için ikili kodları manuel olarak yazmak zorundaydılar. Bu, hataya açık ve verimsiz bir süreçti.

Sembolik Diller (Assembly Languages):

Makine dilinin zorlukları nedeniyle, **sembolik diller** ortaya çıktı. Sembolik diller, makine diline daha yakın olmakla birlikte, ikili kodları sembollerle (kelimelerle) temsil eder. Bu diller, programcıların yazdığı komutları, daha anlaşılır sembollerle ifade etmelerini sağladı.

Özellikleri:

Makine diline göre biraz daha anlaşılırdır, çünkü her makine komutu bir sembolik komutla temsil edilir.

Her işlemci tipi için farklı bir sembolik dil vardır (yani, **düşük seviyeli dil** olarak kabul edilir).

Programcılar **assembler** adı verilen bir program kullanarak sembolik komutları makine diline dönüştürürler.

Tarihsel Gelişim:

Sembolik diller, bilgisayarın düşük seviyeli komutlarına daha kolay erişim sağlar. Bu dillerin ortaya çıkışı, yazılımlarının daha hızlı ve verimli bir şekilde yazılabilmesini sağladı. Ancak, yine de program yazmak zordu ve hata yapma olasılığı yüksekti.

Yüksek Seviyeli Diller (High-Level Languages):

Yüksek seviyeli diller, bilgisayarlar için anlaşılır olan komutları daha doğal dillerde, yani insanların daha kolay anlayabileceği biçimde yazılabilmesini sağlayan dillerdir. Bu diller, işlemciyi ve makine dilini arka planda yönetir. Programcılar, çok daha basit ve anlaşılır komutlarla yazılım geliştirebilirler.

Özellikleri:

İnsan dostu ve anlaşılması kolaydır, çünkü doğal dillere (örneğin, İngilizce) benzer yapılar kullanır.

Programcılar, bu dillerle yazılım geliştirirken donanımın karmaşıklığı ile uğraşmazlar.

Yüksek seviyeli diller, genellikle **derleyiciler** veya **yorumlayıcılar** kullanarak makine diline çevrilir.

Tarihsel Gelişim:

Yüksek seviyeli dillerin gelişimi, yazılım geliştirmeyi büyük ölçüde kolaylaştırdı. İlk yüksek seviyeli diller arasında **Fortran** (1950'ler) ve **COBOL** (1960'lar) yer alır. Zamanla, **C**, **Java**, **Python** ve **C++** gibi daha güçlü ve kullanıcı dostu diller ortaya çıkmıştır. Bu diller, hem donanım hem de yazılım mühendisliğini daha erişilebilir hale getirdi.

Bilgisayarın Anlayacağı Dil: Tarihsel Bir Bakış

1. İlk Zamanlar - Makine Dili:

1. İlk bilgisayarlar, sadece makine dili ile çalışıyordu. Bu, doğrudan işlemci ile iletişim kurabilen tek dildi.
2. Komutlar 0'lar ve 1'lerden oluşan ikili bir biçimdeydi ve bu dillerin yazılması ve anlaşılması son derece zordu.

2. Sembolik Diller Dönemi:

1. Bilgisayarların gelişmesiyle birlikte, sembolik diller ortaya çıktı. Bu diller, ikili komutları sembollerle ifade etmeye başladı.
2. Programcılar, **assembler** kullanarak bu sembolik dilleri makine diline dönüştürüyordu.

3. Yüksek Seviyeli Diller:

1. 1950'lerden itibaren, daha anlaşılır ve verimli diller geliştirilmeye başlandı. Bu diller, doğal dil yapısına daha yakın komutlarla yazılabiliyordu.
2. Bu dillerle yazılım geliştirme süreci hızlandı ve yazılımlar daha kullanıcı dostu hale geldi.

4. Modern Zamanlar - Bugünün Yüksek Seviyeli Dilleri:

1. Günümüzde, Python, JavaScript, C++, Java gibi diller, yazılım geliştiricilerin daha verimli ve hızlı bir şekilde kod yazmalarını sağlar. Bu diller, donanımın karmaşıklığını gizler ve geliştiricilere daha verimli bir çalışma ortamı sunar.

Algoritmik Yaklaşımda C Dili Esnekliği ve Özellikleri

C programlama dili, hem üst düzey hem de alt düzey programlama dillerinin özelliklerini birleştirerek, esnek ve güçlü bir dil yapısı sunar. UNIX/LINUX gibi işletim sistemlerinin temel dilidir. Algoritma tasarımı, sistem konfigürasyonu ve veri modeli oluşturma gibi alanlarda yoğun olarak kullanılır. C dili, doğrudan bellek adresleriyle çalışabilme, yeni veri yapıları tanımlayabilme, bit düzeyinde işlem yapabilme, disk dosyalarına erişim ve işletim sistemi çağrılarını kullanma gibi özelliklerle esneklik sağlar. Bu özellikleri sayesinde çok geniş bir uygulama yelpazesinde, endüstriyel kontrol uygulamalarından internet ve ağ programlamasına kadar tercih edilmektedir. C dilinin sunduğu fonksiyonel kütüphaneler, bellek yönetimi, işaretçiler ve fonksiyonel esneklik gibi özellikler, program geliştirmeyi kolaylaştırır.

C++

C diline nesne yönelimli programlamayı (OOP) ekleyerek C++ dili geliştirilmiştir. Bjarne Stroustrup, 1979 yılında "C with Classes" adıyla ilk versiyonunu geliştirdi. Daha sonra, 1980'li yılların başında bu dil "C++" olarak adlandırıldı.

C++, C dilinin güçlü özelliklerini koruyarak nesne yönelimli programlamaya (OOP) yönelik özellikler ekler. Bu eklenen özellikler şunlardır:

- Sınıflar (Classes)
- Kapsülleme (Encapsulation)
- Miras (Inheritance)
- Polimorfizm (Polymorphism)
- Soyutlama (Abstraction)
- Çoklu Kalıtım (Multiple Inheritance)
- Şablonlar (Templates)
- Operatör Aşırı Yükleme (Operator Overloading)
- Metot Aşırı Yükleme (Function Overloading)
- Dinamik Bellek Yönetimi (Dynamic Memory Management)
- Sanal Fonksiyonlar (Virtual Functions)
- Referanslar (References)
- Akıllı Göstergeler (Smart Pointers)
- Exception Handling (Hata Yönetimi)
- Lambda Fonksiyonları (Lambda Functions)
- C++ Standart Kütüphanesi (STL)

•Sınıflar (Classes):

C dilinde yer almayan, C++ dilinde nesneler oluşturmak için kullanılan sınıflar (classes) eklenmiştir. Sınıflar, nesnelerin veri (özellikler) ve bu verilere uygulanabilecek işlemleri (fonksiyonlar/metotlar) bir arada tutar. Sınıflar, nesne yönelimli programlamanın temel yapı taşıdır.

•Kapsülleme (Encapsulation):

Kapsülleme, verilerin ve davranışların gizlenmesi prensibine dayanır. Bu sayede sınıfın iç işleyişi dış dünyadan soyutlanır ve dışarıdan yalnızca belirlenen yöntemlerle erişilebilir. Bu, veri güvenliğini artırır ve yazılımın bakımını kolaylaştırır.

•Miras (Inheritance):

C++ dilinde bir sınıf, başka bir sınıftan özellik ve davranışları miras alabilir. Miras, yazılımda kodun tekrar kullanılabilirliğini artırır, yeni sınıflar eski sınıfların işlevselliğini devralarak daha hızlı ve verimli geliştirme yapılmasını sağlar.

•Polimorfizm (Polymorphism):

Polimorfizm, aynı isimdeki fonksiyonların veya metotların farklı nesneler üzerinde farklı şekillerde çalışabilmesidir. Bu, esnek ve genişletilebilir yazılım yapıları oluşturulmasına olanak tanır ve özellikle soyutlama işlemi için çok önemlidir.

•Soyutlama (Abstraction):

Soyutlama, karmaşık sistemlerin sadece önemli özelliklerinin kullanıcılara sunulmasıdır. C++'ta soyut sınıflar ve sanal metotlar kullanılarak, belirli bir işlevselliğin nasıl çalıştığı değil, ne yaptığı hakkında bilgi verilir. Bu, yazılımın daha anlaşılır ve bakımı kolay hale gelmesini sağlar.

•Çoklu Kalıtım (Multiple Inheritance):

C++, bir sınıfın birden fazla sınıftan miras almasına olanak tanır. Bu özellik, bir nesneye birden fazla türde davranış kazandırılmasını sağlar. Çoklu kalıtım, kodun daha esnek ve modüler olmasına yardımcı olur.

•Şablonlar (Templates):

C++ dilinde, fonksiyonlar ve sınıflar tür bağımsız bir şekilde şablonlar olarak tanımlanabilir. Şablonlar, aynı fonksiyon ya da sınıfın farklı türlerdeki verilerle çalışabilmesine olanak tanır. Bu, tür güvenliğini sağlarken esnekliği artırır.

•Operator Overloading (Operatör Aşırı Yükleme):

C++ dilinde, operatörler kendi özel işlevlerini gerçekleştirecek şekilde aşırı yüklenebilir. Bu, kullanıcı tanımlı türler (örneğin sınıflar) için daha anlaşılır ve doğal bir sözdizimi sağlar.

•Metot Aşırı Yükleme (Function Overloading):

Aynı isimdeki metotlar, farklı parametrelerle tanımlanarak aşırı yüklenebilir. Bu, metotları daha esnek hale getirir ve aynı işlevi farklı türlerdeki verilerle çalışacak şekilde tanımlamayı mümkün kılar.

•Dinamik Bellek Yönetimi (Dynamic Memory Management):

C++ dilinde dinamik bellek tahsisi ve serbest bırakılması programcı tarafından kontrol edilir. Bu, yazılımın bellek kullanımını optimize etmeye ve belleği verimli bir şekilde yönetmeye olanak tanır. new, delete, malloc, free gibi komutlar ile bellek yönetimi yapılabilir.

•Sanal Fonksiyonlar (Virtual Functions):

C++'ta sanal fonksiyonlar, türemiş sınıflarda farklı şekilde davranacak şekilde yeniden tanımlanabilir. Bu, dinamik bağlama (dynamic binding) sağlar ve polimorfizmi etkin bir şekilde kullanmayı mümkün kılar.

•Referanslar (References):

C++ dilinde, deęişkenlerin doğrudan adresine referanslar oluşturulabilir. Bu, bellek kullanımını optimize eder ve daha hızlı veri aktarımı sağlar.

•Akıllı Göstergeler (Smart Pointers):

C++11 ile tanıtılan akıllı göstergeler, bellek sızıntılarını engellemek için dinamik bellek yönetimini daha güvenli hale getirir. `std::unique_ptr`, `std::shared_ptr`, ve `std::weak_ptr` gibi akıllı göstergeler ile bellek güvenliği sağlanır.

•Exception Handling (Hata Yönetimi):

C++ dilinde, `try`, `catch`, ve `throw` anahtar kelimeleri ile hata yönetimi yapılabilir. Bu, programın hata durumunda düzgün bir şekilde çalışmaya devam etmesini sağlar ve daha güvenli kod yazılmasına olanak tanır.

•Lambda Fonksiyonları (Lambda Functions):

C++11 ile gelen lambda fonksiyonları, fonksiyonları kısa ve anonim bir şekilde tanımlamayı sağlar. Bu, özellikle koleksiyonlar ve algoritmalar ile çalışırken büyük kolaylık sağlar.

•C++ Standart Kütüphanesi (STL):

C++'ta, verimli ve güçlü veri yapıları ile algoritmalar içeren geniş bir standart kütüphane bulunur. `vector`, `list`, `map`, `set` gibi veri yapıları ve `sort()`, `find()` gibi algoritmalar, yazılım geliştirme sürecini hızlandırır.

Bu maddeler, C++ dilinin temel ve önemli özelliklerini özetler ve nesne yönelimli programlamanın yanı sıra dilin esnekliğini, verimliliğini ve gücünü vurgular. C++, güçlü bellek yönetimi, esnek yapılar ve yüksek performansı ile yazılım geliştirmede yaygın bir tercih edilen dil haline gelmiştir.

Nesne Teknolojisine Giriş

Yeni ve daha güçlü yazılımlara talepler artarken hızlı doğru ve ekonomik bir şekilde yazılım geliştirmek ulaşılması zor bir hedef gibi gelmektedir. Nesneler veya daha doğrusu nesnelerin geldiği sınıflar tekrar kullanılabilen yazılım bileşenleridir. Tarih nesnesi, ses nesnesi, zaman nesnesi, video nesnesi, otomobil nesnesi, insan nesnesi vb. nesneler bulunmaktadır. İsim verebildiğimiz her şey özellik ve davranış olarak bir yazılım nesnesi ile temsil edilebilir. Yazılım geliştiricileri nesneye yönelimli programlama tekniği ile yazılım geliştirme gruplarının önceki tekniklere göre çok daha üretken çalışabildiğini keşfetmişlerdir. Nesneye yönelik programlar genelde daha kolay anlaşılmakta ve değiştirilebilmektedir.

Bir Nesne Olarak Otomobil

Basit bir kıyaslama ile başlayalım. Düşünün ki bir araba sürmek istiyorsunuz ve arabanın daha hızlı gidebilmesi için gaz pedalına basmalısınız. Bunun için ilk önce ne yapılması gerekir? Bir arabayı kullanabilmek için öncelikle birisinin arabayı tasarlaması gerekir. Araba genellikle bir mühendisin çizimleriyle başlar. Bu bir evin tasarımını tanımlayan planlara benzemektedir. Çizimler içerisinde gaz pedalının tasarımı da bulunacaktır.

Pedal arabanın hızlı gidebilmesi için devreye giren karmaşık mekanizmayı sürücüden gizlemektedir aynı şekilde fren pedalı arabayı yavaşlatan ve direksiyonda arabaya yön veren karmaşık mekanizmayı sürücüden gizlemektedir. Bu sayede sürücüler motorun nasıl çalıştığı, yavaşladığı veya döndürme mekanizmasını nasıl işlediğini bilmez ve nasıl işlediği hakkında hiçbir bir bilgi olmadan otomobili kolayca kullanır. Arabayı kullanabilmemiz için mühendisinin çizimlerindeki gibi bir arabanın üretilmesi gerekir. Yapılmış tam bir otomobilin aracı hızlandıracak gerçek bir pedalı bulunur. Fakat bu yeterli değildir. Çünkü araba kendi kendine hızlanamaz yani arabanın hızlanabilmesi için sürücünün pedala basması gerekir.

Üye Fonksiyonlar ve Sınıflar

Araba örneğimizi kullanarak nesneye yönelimli programlamanın temel kavramlarını tanıyalım. Program içerisinde bir görev yapabilmek için bir üye fonksiyonuna ihtiyacımız vardır. Üye fonksiyon görevini gerçekleştiren program ifadelerine sahiptir. Gaz pedalının otomobili hızlandıran mekanizmayı sakladığı gibi üye fonksiyonda program ifadelerini kullanıcısından saklamaktadır.

C++ dilinde sınıf adı verilen program birimi kendisine ait görevleri gerçekleştirecek üye fonksiyonları barındıran yapıdır.

(Örneğin bir banka hesabını temsil eden bir sınıfın üye fonksiyonlarından birisi hesaba para yatırırken bir değeri hesaptan para çekebilir ve üçüncüsü de hesabın anlık dengesini sorgulayabilir.)

Örnek Oluřturma

Nasıl arabayı sürebilmek için öncelikle birisinin arabayı tasarım çizimlerinden inşa etmesi gerekiyorsa aynı şekilde sınıfa ait görevleri gerçekleřtirebilmek için de sınıftan bir nesne inşa etmemiz gerekir. Bu işleme örnek oluřturma adı verilir ve oluřturulan nesne örnek şeklinde nitelendirilir.

Tekrar Kullanılabilirlik Özelliđi

Arabaya ait olan çizim birçok arabanın üretiminde kullanılabileceđi gibi bir sınıfta birden fazla nesne oluşturmak için tekrar kullanılabilir. Var olan sınıfları, yeni programlar ve sınıflar oluştururken tekrardan kullanmak zamanla iş gücünden kazanç sağlayacaktır. Daha önceden tasarlanmış sınıflar birçok testten ve geliştirme işleminden geçtiđi için bu sınıfları yeniden kullanmak daha güvenilir ve verimli sistemler inşa etmemizi sağlayacaktır. Nasıl deđiştirilebilir parçalar sanayi devriminin önemli bir parçası ise tekrar kullanılabilir sınıflar yazılım devriminin önemli bir parçasıdır.

Mesajlar ve Üye Fonksiyon Çağrıları

Araba sürerken gaz pedalına basmak arabaya bir görev yapması için mesaj yollamaktır. Aynı şekilde bir nesneye de mesaj yollanabilir. Her bir mesaj bir üye fonksiyon çağrısı şeklinde gerçekleşmektedir. Bu çağrı nesneye ait üye fonksiyonun görevini yapmasını söylemektedir.

Dr Öğr. Üyesi Fatma AKALIN

Özellikler ve Veri Üyeleri

Bir araba çeşitli görevleri yapabilme yeteneğine sahip olduğu gibi renk, kapı sayısı, depodaki gaz miktarı, anlık hız ve toplam aldığı yol gibi özelliklere sahiptir. Yetenekleri gibi özellikleri de mühendis çizimi içerisinde yer alacaktır. Gerçek bir arabaya sürdüğünüzde bu özelliklerde arabayla beraber hareket edecektir. Her bir araba kendi özelliklerini kontrol etmektedir. Örneğin her bir araba kendi tankındaki gaz miktarını bilmektedir. Fakat diğer arabalardaki bu bilgi hakkında öngörüler yoktur. Nesnelerde benzer şekilde özelliklere sahiptirler. Aynı araba örneğinde olduğu gibi her nesne program içerisinde kendi özelliklerini kullanmaktadır. Bu özellikler nesnelerin ait olduğu sınıf tasarımında belirtilmektedir. Özellikler sınıfın veri üyeleri olarak tanımlanmaktadır.

Kapsülleme

Sınıflar, özellik ve üye fonksiyonları nesneler içerisine kapsüller. Bir nesnenin özelliği ile üye fonksiyonu arasında yakın bir ilişki bulunmaktadır. Nesneler birbirleri arasında iletişim kurabilirler. Fakat nasıl üretildikleri hakkında bir bilgileri yoktur. Üretilme bilgisi nesneler içerisinde saklanmıştır. Bu bilgi gizleme yazılım mühendisliğinin önemli bir parçasıdır.

KALITIM

Yeni nesnelerin sınıfları kalıtım yoluyla hızlı ve rahat bir şekilde oluşturulabilir. Bu yöntemde yeni sınıf var olan bir sınıfın bütün özelliklerini içine alır. Kimi durumlarda ise bu özellikler yeni sınıfta değiştirilir ve kendine has bir karakteristik verilir. Araba örneğimize dönersek üstü açılır araba genel bir araba sınıfı değildir. Bu yeni sınıf arabaların çatısı açılıp kapanabilmektedir.

```
class Araba {  
public:  
    int hiz;  
  
    Araba() { hiz = 0; } // Kurucu fonksiyon  
  
    void hizlan(int artis) {  
        hiz += artis;  
    }  
};  
  
int main() {  
    Araba benimArabam; // Araba sınıfından bir nesne oluşturma (örnekleme)  
    benimArabam.hizlan(20); // Üye fonksiyonunu çağırma  
}
```

Üye Fonksiyon (Member Function)

- Bir sınıfın içinde tanımlanan fonksiyonlara **üye fonksiyon** denir.
- Örneğin, aşağıdaki hizlan fonksiyonu **Araba sınıfının bir üye fonksiyonudur** çünkü doğrudan sınıfın içinde tanımlanmıştır:

Örnek (Instance) Oluşturma

- Bir sınıftan nesne oluşturmak, **örnek (instance) oluşturma** işlemidir.
- Örneğin, aşağıdaki kod Araba sınıfından bir nesne oluşturur:


```
class Araba {  
protected: // "private" yerine "protected" kullandık ki alt sınıflar erişebilsin  
    int hiz;  
  
public:  
    Araba() { hiz = 0; }  
  
    void setHiz(int yeniHiz) {  
        if (yeniHiz >= 0) {  
            hiz = yeniHiz;  
        }  
    }  
  
    int getHiz() {  
        return hiz;  
    }  
};
```

Burada neyi kapsüllüyoruz?

- hiz değişkenini dışarıdan doğrudan erişime kapatıyoruz.
- Sadece setHiz() ve getHiz() fonksiyonları aracılığıyla güvenli bir şekilde değiştirilmesine ve okunmasına izin veriyoruz.

Türetilmiş Sınıf (Derived Class)

Dr. Öğr. İ.

```
class SporAraba : public Araba { // Araba sınıfından kalıtım alıyoruz
public:
    void nitro() { // Nitro özelliği ekledik (hızı artırıyor)
        hiz += 50; // "hiz" protected olduğu için erişilebilir
    }
};
```

ALIN

Kullanım (Main Fonksiyonu)

Dr. Öğr. Üyesi

```
int main() {  
    SporAraba ferrari;  
  
    ferrari.setHiz(100); // Araba sınıfından gelen üye fonksiyon  
    ferrari.nitro(); // SporAraba sınıfına özgü fonksiyon  
  
    std::cout << "Ferrari'nin hizi: " << ferrari.getHiz() << " km/h" << std::endl;  
  
    return 0;  
}
```

LIN

Çıktı: Ferrari'nin hizi: 150 km/h

```

#include <iostream>

class Araba {
private:
    int hiz; // Kapsülleme: hiz değişkeni dışarıdan erişilemez

public:
    Araba() { hiz = 0; }

    void setHiz(int yeniHiz) {
        if (yeniHiz >= 0) {
            hiz = yeniHiz;
        }
    }

    int getHiz() {
        return hiz;
    }
};

class SporAraba : public Araba {
public:
    void nitro() {
        setHiz(getHiz() + 50); // hiz değişkenine doğrudan erişemiyoruz, kapsülleme var
    }
};

int main() {
    SporAraba ferrari;

    ferrari.setHiz(100); // Araba sınıfından gelen üye fonksiyon
    ferrari.nitro(); // SporAraba sınıfına özgü fonksiyon

    std::cout << "Ferrari'nin hizi: " << ferrari.getHiz() << " km/h" << std::endl;

    return 0;
}

```

Burada hiz değişkeni private olduğu için doğrudan erişilemiyor, fakat setHiz() ve getHiz() fonksiyonları aracılığıyla kontrol ediliyor.

Örneğin:

- setHiz(100); → hiz değişkenini değiştirmek için çağrıldı.
- getHiz(); → hiz değerini almak için çağrıldı.
- nitro(); → setHiz(getHiz() + 50); şeklinde hiz değişkenine dolaylı olarak erişti.

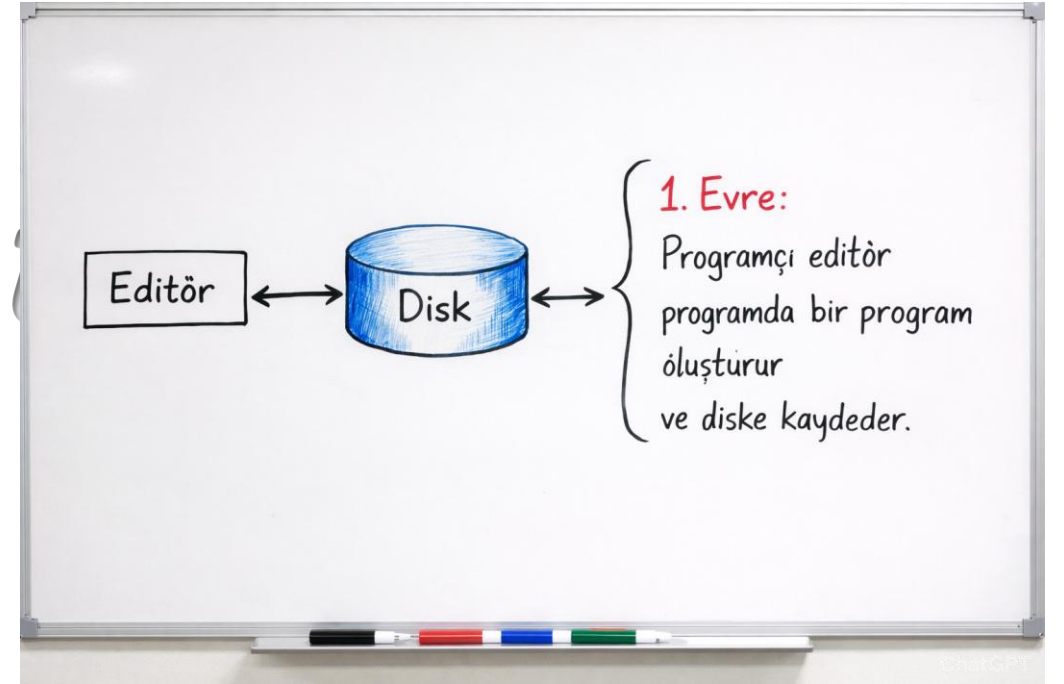
Atma AKALIN

Tipik C++ Program Geliştirme Ortamı

C++ sistemleri genelde 3 bölümden oluşur. Bunlar program geliştirme ortamı, dil ve c++ standart kütüphanesidir. c++ programı genelde 6 evreden geçmektedir. Bu evreler; düzenleme, ön işleme, derleme, bağlama, yükleme ve çalıştırmadır. Şimdi tipik bir C++ program geliştirme ortamını anlatacağız

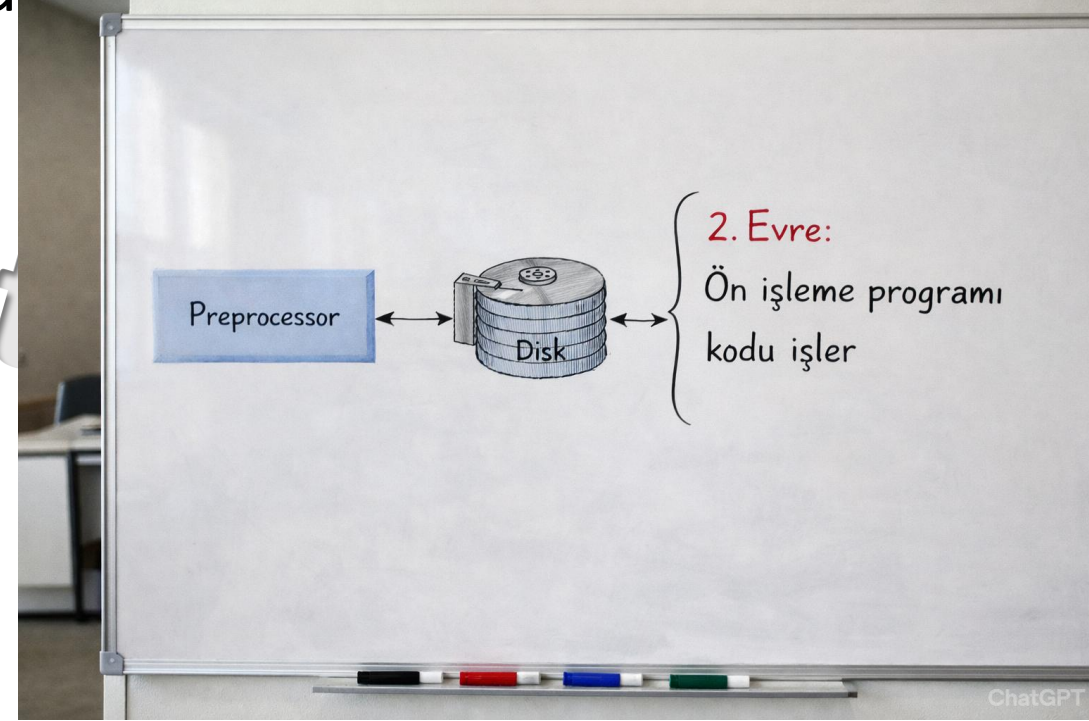
1. Evre: Düzenleme Programı

İlk evre bir dosyanın düzenleme programı ile düzenlenmesine içermektedir. Bu programlar genelde düzenleyici (editör) olarak adlandırılmaktadır. Düzenleyici kullanılarak C++ programlarınızı (kaynak kod olarak adlandırılır) yazabilirsiniz. Ayrıca programınızda gerekli düzenlemeleri yapıp programı ikincil depolama cihazlarına kaydedebilirsiniz. C++ kaynak dosyaları genelde .cpp ile bitmektedir.



2. Evre: Bir C++ Programını Ön İşleme

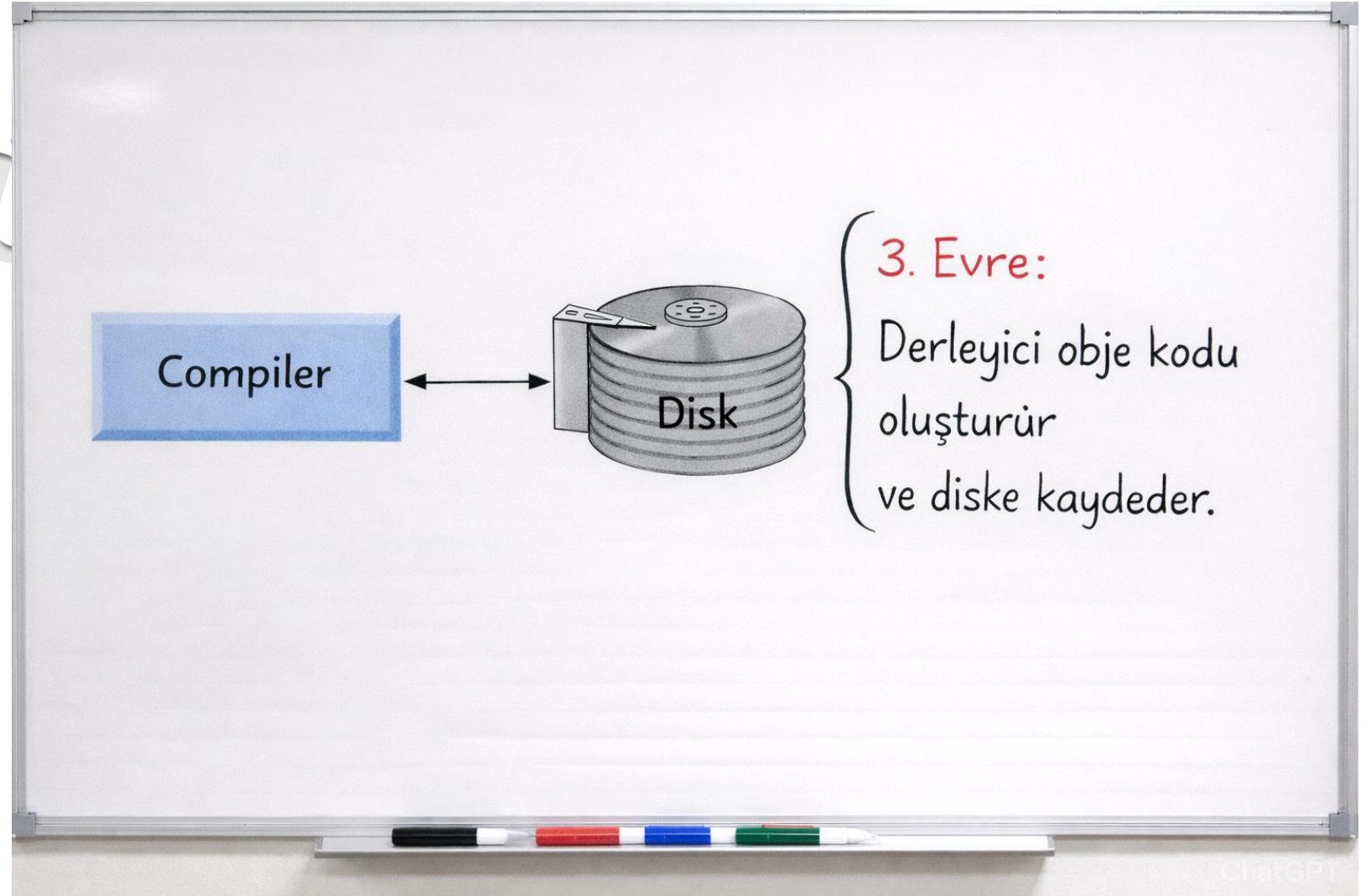
İkinci evrede programınızı derleyecek komutu verirsiniz. C++ sistemlerinde derlemenin çeviri evresi başlamadan «ön işleme» devreye girmektedir Bu yüzden derleme işlemini 3. evre olarak adlandırdık. C++ ön işlemcisi ön işleme direktifleri olarak adlandırılan komutları uygulamaktadır. Bu komutlar derleme işlemi başlamadan önce kaynak kod üzerinde çeşitli manipölasyonlar yapmaktadır. Bunlardan birisi başka bir kod dosyasının derlenmek için eklenmesidir.



3. Evre: Bir C++ Programını Derleme

Dr. Öğr. U.

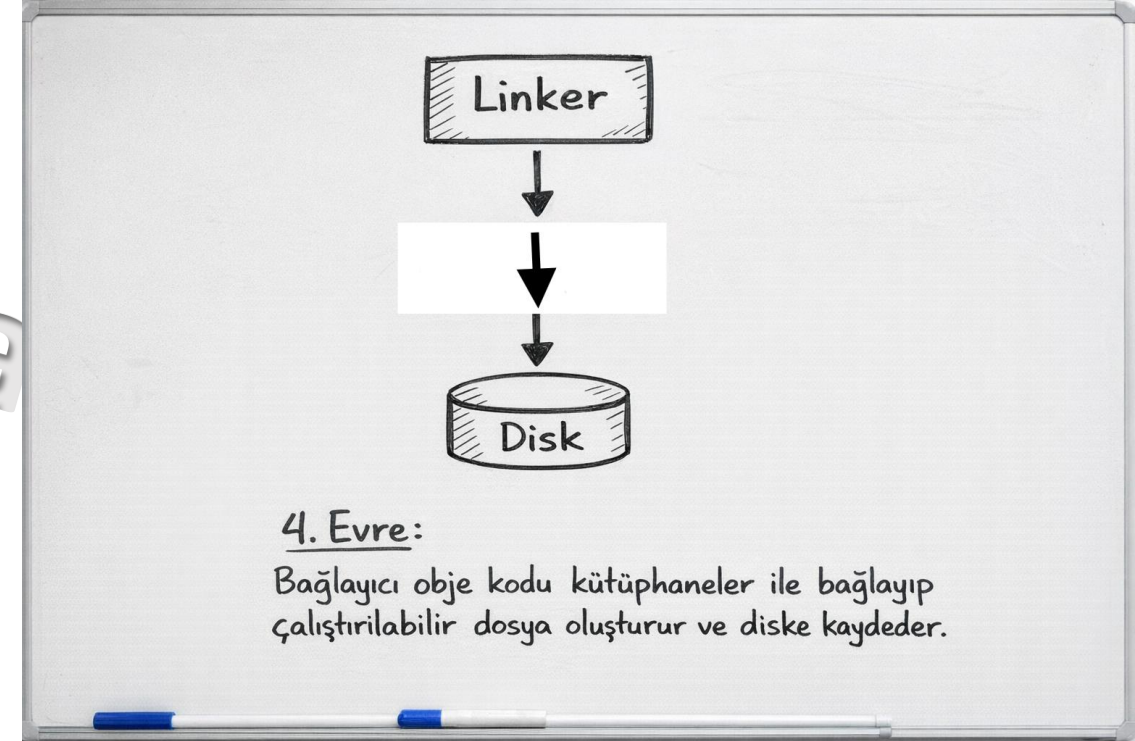
3. evrede
derleyici c++
programını
makine
koduna(obje
koduna olarak da
adlandırılır)
çevirir.



4. Evre: Baęlama

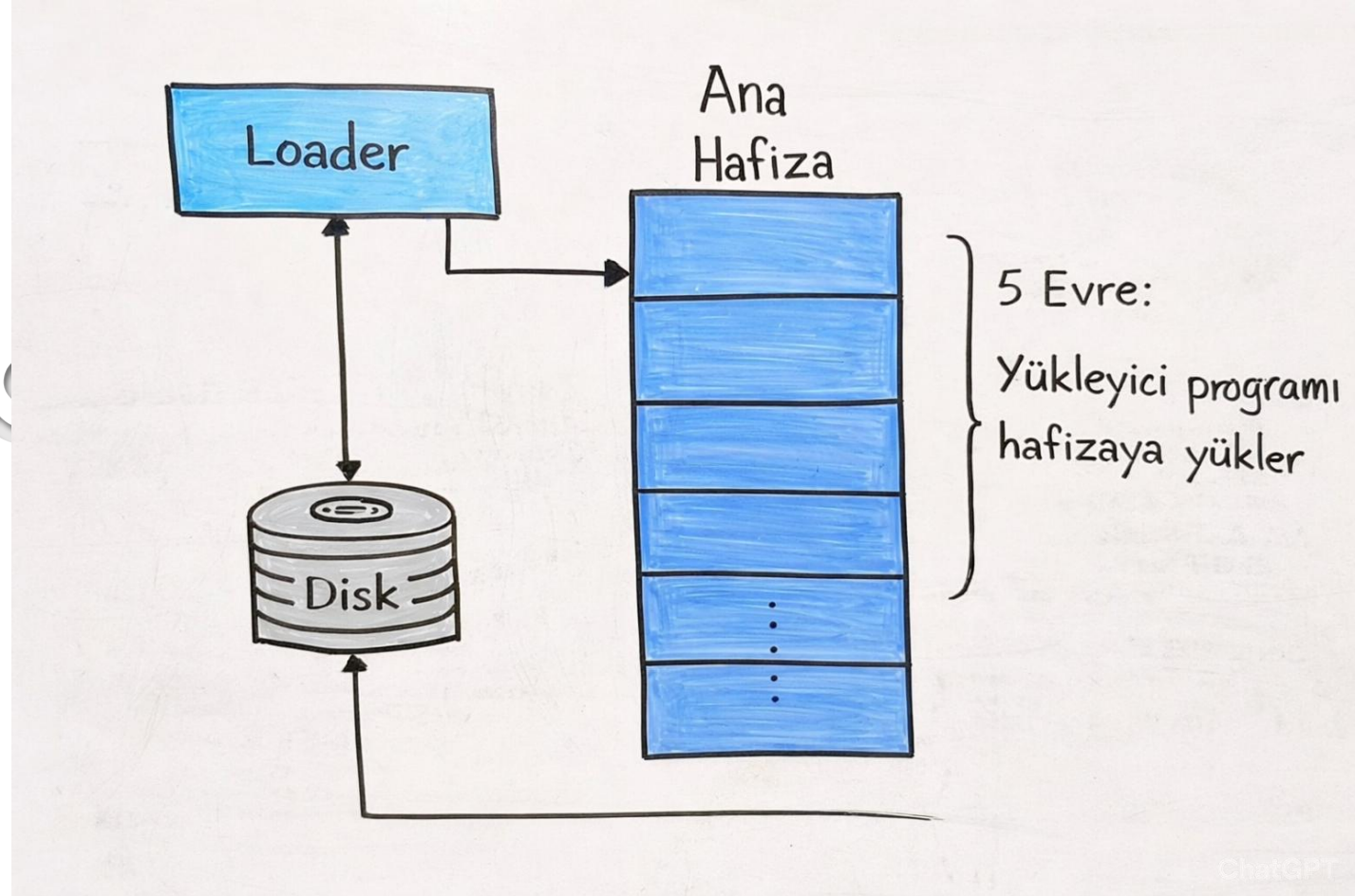
4. evre baęlama olarak adlandırılır. C++ programları genellikle başka yerlerde tanımlanmış fonksiyon veya verileri kullanmaktadır. (Örneęin standart kütüphanelerin kullanılması gibi.). C++ derleyicisi tarafından oluşturulan obje kodu bu eksikler yüzünden boşluklara sahiptir.

Baęlayıcı, obje kodda kullanılmış fakat tanımlanmamış olan fonksiyon veya verileri ait oldukları kütüphaneler ile baęlayarak çalıştırabilir program oluşturmaktadır. Eğer program doğru bir şekilde deęerlendirip baęlanırsa 1 adet çalıştırılabilir dosya oluşacaktır.



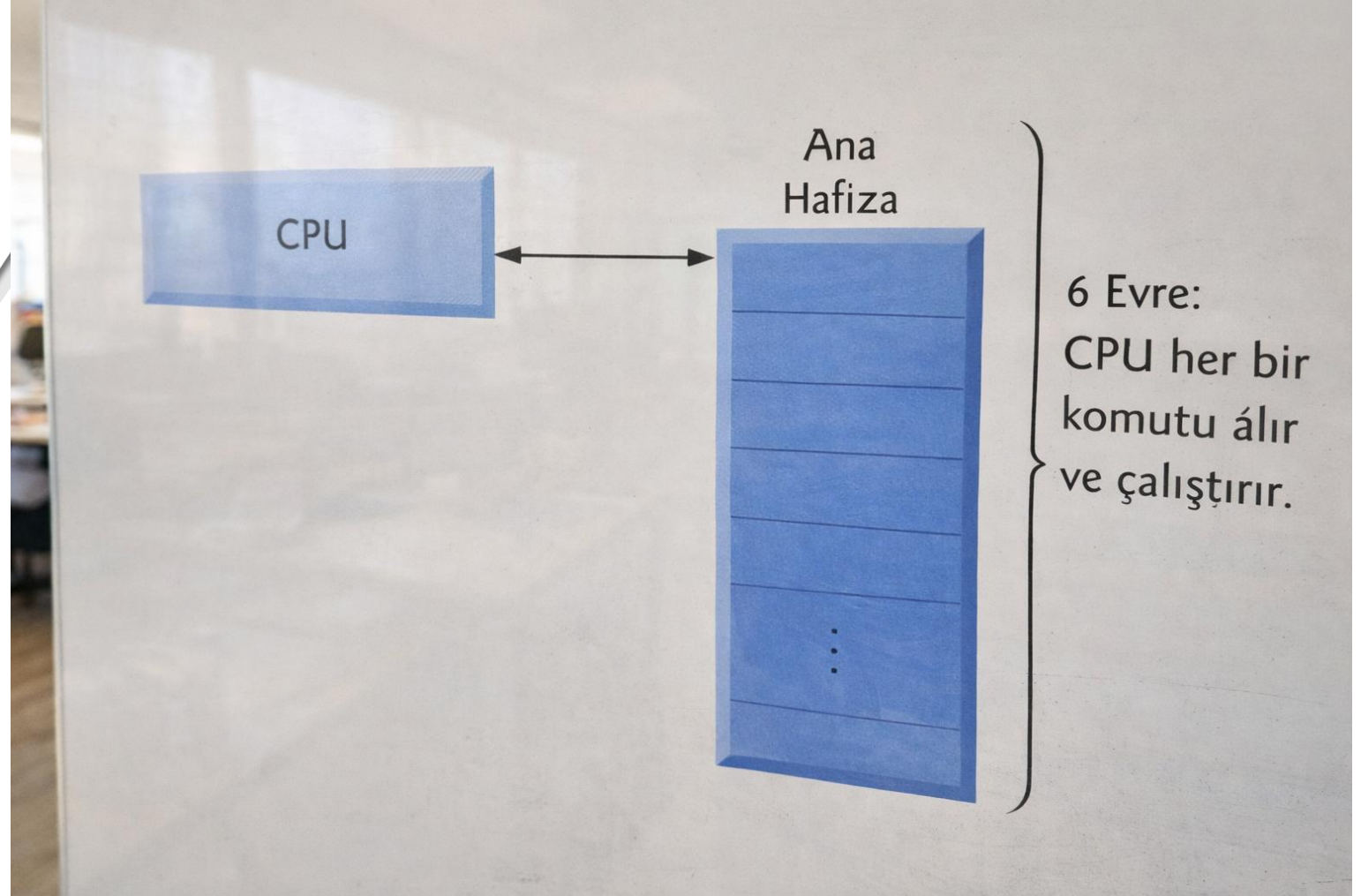
5. Evre: Yükleme

Bir program çalıştırılmadan önce hafızaya yüklenmesi gerekir. Bu evreye yükleme adı verilir. Bu işlem yükleyici tarafından yapılmaktadır. Yükleyici çalıştırılabilir dosyayı sabit diskten alıp hafızaya transfer eder. Program tarafından kullanılan ortak kütüphanelerde hafızaya yüklenecektir

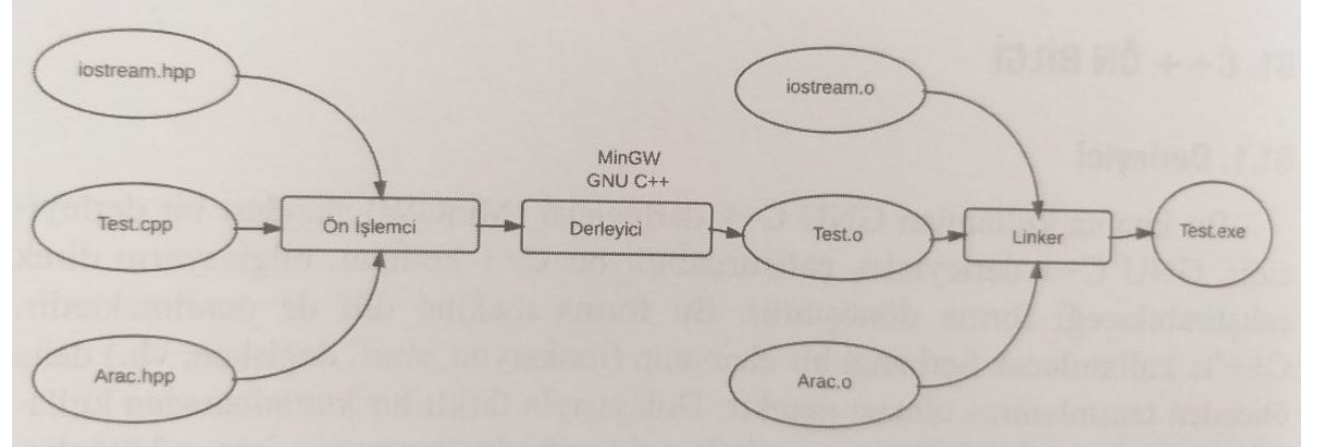
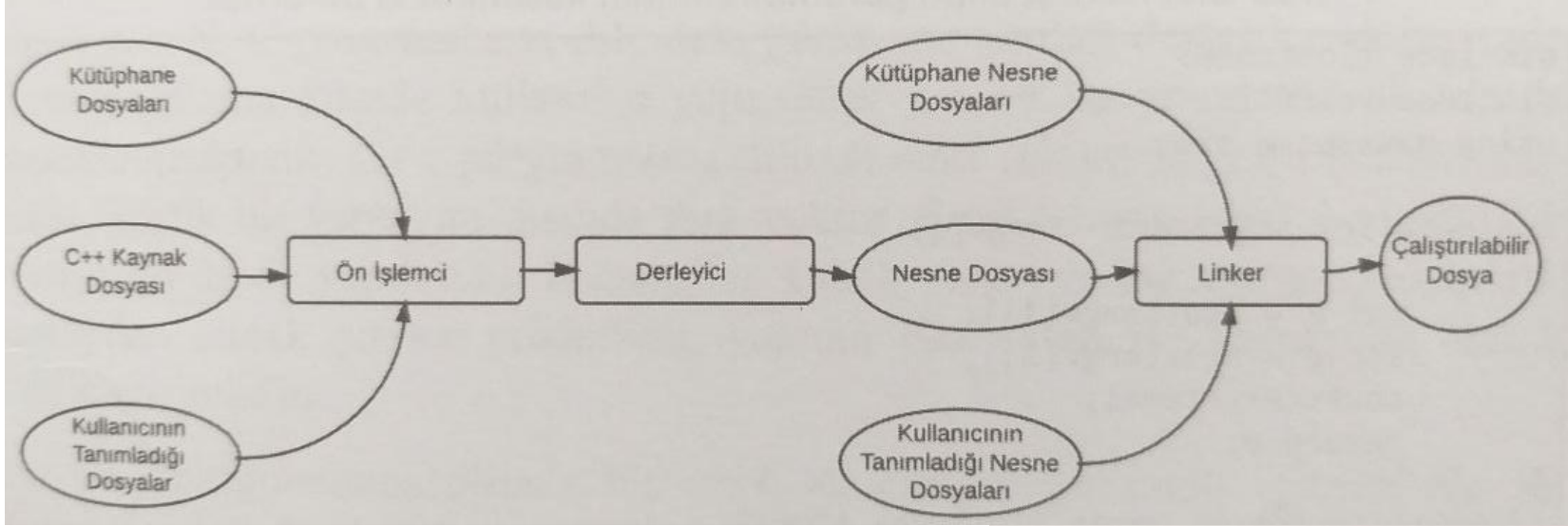


6. evre: Çalıştırma

Son evrede bilgisayar, merkezi işlemci kontrolünde programı bir seferde bir komut işletilecek şekilde çalıştırır. Bazı modern bilgisayar mimarileri aynı anda birden fazla komutu paralel olarak işletebilmektedir



Özetlersek,



GNU C++ derleyicisi (MinGW)

Bu derste .exe dosyası oluşturmak için kullanılacak derleyici, GNU C++ derleyicisi (MinGW) olacaktır.

Sizlere verdiğim ödevlerin Mingw derleyicisi ile derlenmesi gerektiği için kişisel bilgisayarınıza kurulumun sağlanması önem arz etmektedir.

Komut Bileşenleri:

1.g++:

Bu, GNU C++ derleyicisi'nin adıdır. g++, C++ programlarını derlemek için kullanılan bir araçtır. C++ kaynak dosyasını derler ve makine diline çevirir.

2.-o deneme:

Bu, derleme sırasında oluşacak çalıştırılabilir dosyanın adını belirler. -o anahtar kelimesi, çıktıyı (işlemciye yüklenen dosya) özel bir isme yönlendirir. Bu durumda, deneme adıyla bir çalıştırılabilir dosya oluşturulacaktır.

Eğer -o kullanılmazsa, derleyici varsayılan olarak a.out adında bir dosya oluşturur.

3.deneme.cpp:

Bu, derlenecek C++ kaynak dosyasının adıdır. deneme.cpp, C++ programının yazıldığı dosya adıdır. Bu dosya, .cpp uzantısıyla biten ve C++ dilinde yazılmış bir kaynak kodunu içerir.

Derleme Adımları:

1.g++ deneme.cpp komutu, deneme.cpp dosyasını derler ve bir çalıştırılabilir dosya üretir.

2.-o deneme seçeneği ile, bu çalıştırılabilir dosyanın adı deneme olarak belirlenir. Eğer -o kullanmasaydık, derleyici çalıştırılabilir dosyayı varsayılan olarak a.out olarak adlandıracaktı.

Bir Örnek Yapalım..

```
deneme.cpp
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     cout<<"Merhaba";
6     return 0;
7 }
```

C:\Windows\System32\cmd.exe

Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1>g++ -o deneme deneme.cpp

cmd			ders_1 klasöründe ara
Ad	Değiştirme tarihi	Tür	
ornek_1	1.01.2024 14:15	Dosya klasörü	
ornek_2	1.01.2024 14:17	Dosya klasörü	
ornek_2_2	1.01.2024 14:14	Dosya klasörü	
ornek_3	1.01.2024 14:14	Dosya klasörü	
ornek_4	1.01.2024 14:14	Dosya klasörü	
deneme.cpp	6.10.2022 14:26	C++ Kaynak Dosy...	

C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1>g++ -o deneme deneme.cpp

C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1>deneme

Merhaba

C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1>

Dersin akışında mingw derleyici kullanımı detaylı olarak anlatılacaktır.

Makefile

Makefile, C++ gibi dillerde derleme işlemini otomatikleştirmek için kullanılan bir betik dosyasıdır.

Neden Kullanılır?

- **Manuel derleme işlemini kolaylaştırır** → g++ main.cpp -o program gibi uzun komutları her seferinde yazmak yerine tek komutla çalıştırırsın:
- **Bağımlılıkları yönetir** → Büyük projelerde **sadece değişen dosyaları** derleyerek **hız kazandırır**.
- **Hata yönetimi sağlar** → Derleme sırasında oluşan hataları takip etmeyi kolaylaştırır.
- **Çok dosyalı projelerde derleme sırasını belirler**.

C:\Users\Fatma\Desktop\ornek_2\makefile - Notepad++

Dosya Düzen Ara Görünüm Kodlama Diller Ayarlar Araçlar Makrolar Çalıştır
Ekleniler Pencereleler ?

makefile

```
1 program: ornek main
2     g++ ornek.o main.o -o program
3 ornek:
4     g++ -c ornek.c -o ornek.o
5 main:
6     g++ -c main.c -o main.o
```

ornek_2 ornek_2 klasör

Ad	Değiştirme tarihi	Tür
Ekran Alıntısı.JPG	10.01.2024 22:18	JPG Dosyası
main.cpp	10.01.2024 22:16	CPP Dosyası
makefile	27.02.2022 09:38	Dosya
ornek.cpp	26.02.2022 10:19	CPP Dosyası
ornek.hpp	26.02.2022 10:18	HPP Dosyası
program.exe	10.01.2024 22:18	Uygulama

C:\Users\Fatma\Desktop\ornek_2\ornek.hpp - Notepad++

Dosya Düzen Ara Görünüm Kodlama Diller Ayarlar Araçlar Makrola

ornek.hpp

```
1 #ifndef ornek_hpp
2     #define ornek_hpp
3
4
5 #endif
```

C:\Users\Fatma\Desktop\ornek_2\ornek.cpp - Notepad++

Dosya Düzen Ara Görünüm Kodlama Diller Ayarlar Araçlar Mak

ornek.cpp

```
1 #include "ornek.hpp"
```

C:\Users\Fatma\Desktop\ornek_2\main.cpp - Notepad++

Dosya Düzen Ara Görünüm Kodlama Diller Ayarlar Araçlar Mak

main.cpp

```
1 #include <iostream>
2 #include <cstdlib>
3 using namespace std;
4 int main()
5 {
6     cout<<"Merhaba";
7 }
```

1. program: ornek main

Bu satır, **program** hedefinin, **ornek** ve **main** hedeflerinin başarıyla tamamlanmasından sonra oluşturulacağını belirtir. Yani, önce **ornek** ve **main** hedefleri derlenip, ardından **program** hedefi oluşturulacak.

- program**: Sonuçta oluşturulacak çalıştırılabilir dosya (output file).

- ornek** ve **main**: Bu hedefler, **program** dosyasını oluşturmak için ihtiyaç duyulan nesne dosyalarıdır (örneğin, **.O** dosyaları).

2. g++ ornek.o main.o -o program

Bu satır, **ornek.o** ve **main.o** nesne dosyalarını birleştirip (linking) **program** adlı çalıştırılabilir dosyayı oluşturur. Burada:

- g++**: C++ derleyicisi.

- ornek.o** **main.o**: Derlenmiş nesne dosyaları.

- o program**: Çıktı dosyasının adının **program** olarak belirlenmesi.

3. ornek:

Bu satır, **ornek** adlı hedefin nasıl oluşturulacağını açıklar. **ornek** hedefi, **ornek.o** nesne dosyasını oluşturmak için kullanılacaktır.

- g++ -c ornek.cpp -o ornek.o**:

- g++**: C++ derleyicisi.

- C**: Derleme aşamasında sadece nesne dosyası (**.O**) oluşturulacağını belirtir, yani derleme yapılacak ama bağlantı (linking) işlemi yapılmayacak.

- ornek.cpp**: Derlenecek kaynak dosya.

- o ornek.o**: Oluşturulacak nesne dosyasının adı **ornek.o** olacak.

4. main:

Bu satır, **main** adlı hedefin nasıl oluşturulacağını açıklar. **main** hedefi, **main.o** nesne dosyasını oluşturmak için kullanılacaktır.

- g++ -c main.cpp -o main.o**:

- g++**: C++ derleyicisi.

- C**: Derleme aşamasında sadece nesne dosyası (**.O**) oluşturulacağını belirtir.

- main.cpp**: Derlenecek kaynak dosya.

- o main.o**: Oluşturulacak nesne dosyasının adı **main.o** olacak.

Özetle:

İlk olarak, ornek.cpp ve main.cpp dosyaları derlenerek ornek.o ve main.o nesne dosyaları oluşturulur.

Ardından bu iki nesne dosyası, program adlı çalıştırılabilir dosyayı oluşturmak için birleştirilir.

make komutu, bu işlemi otomatik olarak gerçekleştirir ve proje derleme işlemi tamamlanmış olur.

Eğer bir dosyada değişiklik yapıldıysa, sadece o dosya tekrar derlenir ve sonrasında program güncellenir. Bu, büyük projelerde derleme süresini önemli ölçüde hızlandırır.

```
C:\Users\Fatma\Desktop\ornek_2\ornek.hpp - Notepad++
Dosya Düzen Ara Görünüm Kodlama Diller Ayarlar Araçlar Makrolar Çalıştır Eklentiler Pencerele ? +
ornek.hpp x
1 #ifndef ORNEK_HPP
2 #define ORNEK_HPP
3
4 class Ornek {
5 public:
6     void MerhabaYaz ();
7 };
8
9 #endif
```

```
C:\Users\Fatma\Desktop\ornek_2\ornek.cpp - Notepad++
Dosya Düzen Ara Görünüm Kodlama Diller Ayarlar Araçlar Makrolar Çalıştır Eklentiler Pencerele ?
ornek.cpp x
1 #include "ornek.hpp"
2
3 #include <iostream>
4 #include "ornek.hpp"
5
6 void Ornek::MerhabaYaz () {
7     std::cout << "Merhaba, Ornek sınıfı!" << std:
8 }
```

```
C:\Users\Fatma\Desktop\ornek_2\main.cpp - Notepad++
Dosya Düzen Ara Görünüm Kodlama Diller Ayarlar Araçlar Makrolar Çalıştır Eklentiler Pencerele ? +
main.cpp x
1 #include <iostream>
2 #include <cstdlib>
3 #include "ornek.hpp"
4
5 int main() {
6     Ornek ornekObjesi;
7     ornekObjesi.MerhabaYaz (); // MerhabaYaz fonksiyonu
8     return 0;
9 }
```

```
C:\Users\Fatma\Desktop\ornek_2>g++ -c ornek.cpp -o ornek.o
C:\Users\Fatma\Desktop\ornek_2>g++ -c main.cpp -o main.o
C:\Users\Fatma\Desktop\ornek_2>g++ ornek.o main.o -o program
C:\Users\Fatma\Desktop\ornek_2>program
Merhaba, Ornek sınıfı!
C:\Users\Fatma\Desktop\ornek_2>
```

```
makefile
1 program: ornek main
2     g++ ornek.o main.o -o program
3 ornek:
4     g++ -c ornek.cpp -o ornek.o
5 main:
6     g++ -c main.cpp -o main.o
```

> ornek_2_2 ornek_2_2 klas

Ad	Değiştirme tarihi	Tür
Ekran Alıntısı.JPG	10.01.2024 22:29	JPG Dosyası
main.cpp	10.01.2024 22:28	CPP Dosyası
makefile	10.01.2024 22:26	Dosya
ornek.cpp	26.02.2022 10:19	CPP Dosyası
ornek.hpp	26.02.2022 10:18	HPP Dosyası
program.exe	24.02.2025 12:20	Uygulama

```
ornek.hpp
1 #ifndef ornek_hpp
2 #define ornek_hpp
3
4
5 #endif

C:\Users\Fatma\Desktop\ornek_2_2\ornek.cpp - Notepad++
Dosya Düzen Ara Görünüm Kodlama Diller Ayarlar Araçlar Makrolar Çalıştır Eklentiler Pencerele ?

ornek.cpp
1 #include "ornek.hpp"

C:\Users\Fatma\Desktop\ornek_2_2\main.cpp - Notepad++
Dosya Düzen Ara Görünüm Kodlama Diller Ayarlar Araçlar Makrolar Çalıştır Eklentiler Pencerele ?

main.cpp
1 #include <iostream>
2 #include <cstdlib>
3 using namespace std;
4 int main()
5 {
6     cout<<"Merhaba";
7 }
```



> ornek_2_2

ornek_2_2



Sırala ▾



Ad

Değiştirme tarihi



Ekran Alıntısı.JPG

10.01.2024 22:29



main.cpp

10.01.2024 22:28



main.o

24.02.2025 12:23



makefile

10.01.2024 22:26



ornek.cpp

26.02.2022 10:19



ornek.hpp

26.02.2022 10:18



ornek.o

24.02.2025 12:23



program.exe

24.02.2025 12:23

Microsoft Windows [Version 10.0.26100.3194]
(c) Microsoft Corporation. Tüm hakları saklıdır

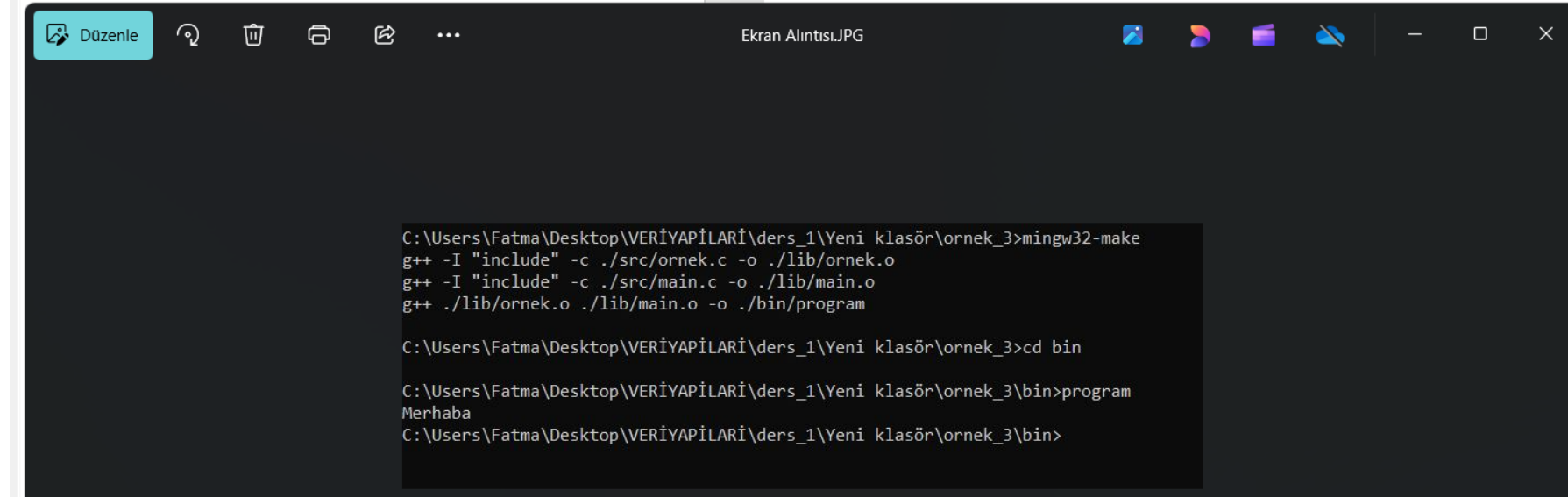
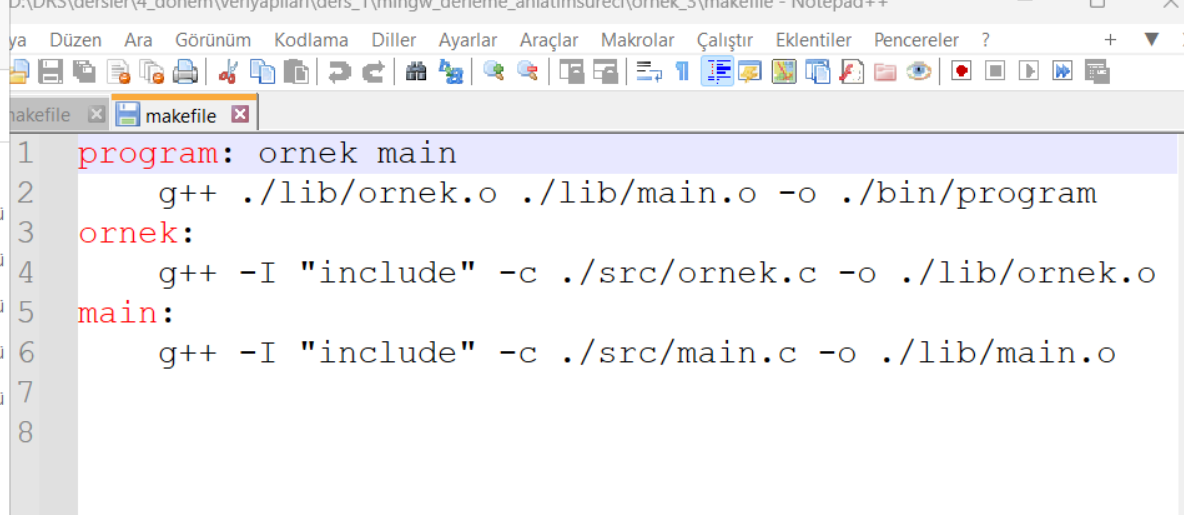
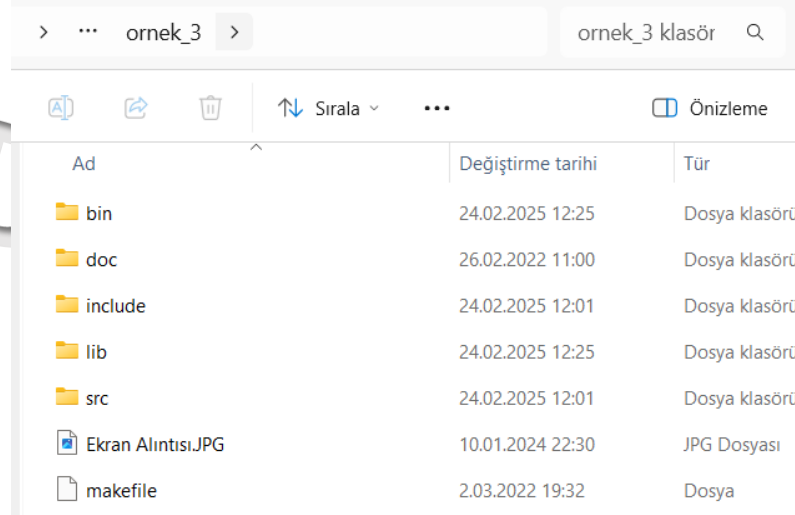
```
C:\Users\Fatma\Desktop\ornek_2_2>mingw32-make  
g++ -c ornek.cpp -o ornek.o  
g++ -c main.cpp -o main.o  
g++ ornek.o main.o -o program
```

```
C:\Users\Fatma\Desktop\ornek_2_2>program
```

Merhaba

```
C:\Users\Fatma\Desktop\ornek_2_2>
```





İlk Satır: Çıktı Programını Oluşturma

program: ornek main

```
g++ ./lib/ornek.o ./lib/main.o -o ./bin/program
```

- program adında bir hedef (target) oluşturuluyor.
- ornek ve main hedeflerine bağımlı, yani önce bunlar derlenmeli.
- g++ ./lib/ornek.o ./lib/main.o -o ./bin/program komutu çalıştırılıyor:
ornek.o ve main.o dosyaları birleştirilerek ./bin/program isimli **çalıştırılabilir dosya** oluşturuluyor.

Ornek Modülünü Derleme

ornek:

```
g++ -I "include" -c ./src/ornek.c -o ./lib/ornek.o
```

- ornek.o dosyasını derlemek için bir hedef.
- g++ -I "include" -c ./src/ornek.c -o ./lib/ornek.o komutu çalıştırılıyor:
- I "include" → include klasörünü başlık dosyaları (header files) için arama dizini olarak kullan.
- c → Sadece nesne dosyası (.o) üret, çalıştırılabilir dosya oluşturma.
- ./src/ornek.c → Kaynak kod dosyası.
- o ./lib/ornek.o → Çıktıyı ./lib/ornek.o dosyası olarak kaydet.

Main Modülünü Derleme

main:

```
g++ -I "include" -c ./src/main.c -o ./lib/main.o
```

- main.o dosyasını derlemek için bir hedef.
- g++ -I "include" -c ./src/main.c -o ./lib/main.o komutu çalıştırılıyor:
- I "include" → include klasörünü başlık dosyaları için kullan.
- c → Sadece nesne dosyası (.o) üret.
- ./src/main.c → Kaynak kod dosyası.
- o ./lib/main.o → Çıktıyı ./lib/main.o olarak kaydet.

ornek_4 klasör

Ad	Değiştirme tarihi	Tür
bin	24.02.2025 12:01	Dosya klasörü
doc	26.02.2022 11:00	Dosya klasörü
include	24.02.2025 12:01	Dosya klasörü
lib	24.02.2025 12:01	Dosya klasörü
src	24.02.2025 12:01	Dosya klasörü
Ekran Alıntısı.JPG	10.01.2024 22:31	JPG Dosyası
makefile	26.02.2022 11:18	Dosya

```
1 derlecalistir: program calistir
2 program: ornek main
3     g++ ./lib/ornek.o ./lib/main.o -o ./bin/program
4 ornek:
5     g++ -I "include" -c ./src/ornek.c -o ./lib/ornek.o
6 main:
7     g++ -I "include" -c ./src/main.c -o ./lib/main.o
8 calistir:
9     ./bin/program.exe
```

Ekran Alıntısı.JPG

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\Yeni klasör\ornek_4>mingw32-make
g++ -I "include" -c ./src/ornek.c -o ./lib/ornek.o
g++ -I "include" -c ./src/main.c -o ./lib/main.o
g++ ./lib/ornek.o ./lib/main.o -o ./bin/program
./bin/program.exe
Merhaba
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\Yeni klasör\ornek_4>
```

C++ Sınıf Yapısı

-C++'da sınıf (class) temel bir yapıdır ve C++'ı C'den ayıran en önemli özelliklerden biridir. C dili, yordamsal (procedural) programlama paradigmasını takip ederken, C++ nesne yönelimli programlamayı (OOP - Object-Oriented Programming) destekler. C dilinde nesne benzeri bir yapı oluşturmak için struct kullanılabilir, ancak struct içine doğrudan fonksiyonlar tanımlanamaz. Bu nedenle, C dili sınıf kavramına sahip değildir, ancak struct ile sınıfa benzer bazı özellikler elde edilebilir.

-C++'da class yapıları struct'tan daha gelişmiş olup, nesne yönelimli programlamaya olanak tanır. C++'da bir sınıf içinde değişkenler, fonksiyonlar, göstericiler ve başka sınıflar gibi farklı üyeler olabilir. Varsayılan olarak, class içinde tanımlanan üyeler private erişimlidir. struct içinde tanımlanan üyeler ise varsayılan olarak public erişimlidir.

Bir C++ sınıfında elemanlara 3 farklı tipte tanımlayıcı getirilebilir. Bunlar;

public: Sınıf dışından sınıfın nesnesi oluşturularak erişilebilen alanlardır.

private: Sınıf dışından hiçbir şekilde erişilemeyen alanlardır.

protected: Sadece sınıf içerisinde ve sınıfı kalıtlayan diğer sınıfların erişebildiği alanlardır.

Bir sınıfın 2 önemli metodu vardır. Bunlar yapıcı ve yıkıcı metotlardır.

Yapıcı metot: Bir sınıftan **nesne oluşturulacağı zaman ilk çağrılan** metottur. Bir sınıfta birden çok yapıcı metot bulunabilir.

Yıkıcı metot: Bir sınıfta oluşturulan nesnelerin **bellege geri döndürülmesi için çağrılan** metottur. Bir sınıfta sadece bir adet bulunabilir.

Bir sınıf birden çok şekilde oluşturulabilir fakat tek bir şekilde yıkılabilir.

Konu Başlıkları

3.1 Giriş

3.2 Üye Fonksiyonlu Sınıf Tanımlama

3.3 Parametrelili Bir Üye Fonksiyon Tanımlama

3.4 Üye Fonksiyon, Set Üye Fonksiyon ve Get Üye Fonksiyon

3.5 Kurucu Fonksiyonlar ile Nesnelere İlk Değer Atama

3.6 Yeniden Kullanılması İçin Sınıfı Ayrı Bir Dosyaya Yerleştirme

3.7 Arayüzü Gerçekleştirmeden Ayırma

3.8 Set Fonksiyonu ile Verinin Doğruluğunun Denetlenmesi

1-Üye Fonksiyonlu Sınıf Tanımlama

kodlar

--C++ ile

programlama,

Dokuzuncu

baskıdan çeviri,

Palme Yayınları,

Pau Deitel,Harvey

Deitel, Çeviri

Editörü Cemil Öz–

Kitabından

alınmıştır.

1-Üye Fonksiyonlu Sınıf Tanımlama

```
main.cpp
1  #include <iostream>
2  using namespace std;
3  class GradeBook{
4  public:
5      void displayMessage() const{
6          cout<<"Welcome to Gradebook!"<<endl;
7      }
8
9  };
10
11 int main() {
12     GradeBook myGradeBook;//myGradeBook isimli
13     //GradeBook sınıfı nesnesi oluşturma
14     myGradeBook.displayMessage();
15 }
```

a AKALIN

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_2>g++ -o main main.cpp
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_2>main
Welcome to Gradebook!
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_2>
```

2-Parametrelili Bir Üye Fonksiyon Tanımlama

```
main.cpp x
1  #include <iostream>
2  using namespace std;
3  class GradeBook{
4  public:
5      void displayMessage(string courseName) const{
6          cout<<"Welcome to Gradebook!\n"<<courseName;
7      }
8
9  };
10
11 int main() {
12     string nameOfCourse;
13     GradeBook myGradeBook;//myGradeBook isimli
14     //GradeBook sınıfı nesnesi oluşturma
15     cout<<"Kurs ismi gir"<<endl;
16     getline(cin,nameOfCourse);
17     cout<<endl;
18     myGradeBook.displayMessage(nameOfCourse);
19 }
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_3>g++ -o main main.cpp
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_3>main
```

```
Kurs ismi gir
```

```
fatma
```

```
Welcome to Gradebook!
```

```
fatma
```

3-Üye Fonksiyon, set Üye Fonksiyon ve get Üye Fonksiyon

```
1 #include <iostream>
2 using namespace std;
3 class GradeBook{
4 public:
5     void setCourseName(string name){
6         courseName=name;//ders ismini nesnede saklama
7     }
8     string getCourseName() const{
9         return courseName;//nesnenin courseName değişken değerini döndürür
10    }
11    void displayMessage()const{
12        cout<<"Hello "<<getCourseName();
13        cout<<endl;
14    }
15 private:
16     string courseName; //GradeBook ders ismi
17 };
18
19 int main(){
20     string nameOfCourse;
21     GradeBook myGradeBook;//myGradeBook isimli
22     //GradeBook sınıfı nesnesi oluşturma
23     cout<<"Kurs ismi "<<myGradeBook.getCourseName()<<endl;
24     cout<<"Baska bir kurs ismi gir"<<endl;
25     getline(cin,nameOfCourse);
26     cout<<endl;
27     myGradeBook.setCourseName(nameOfCourse);
28     cout<<endl;
29     myGradeBook.displayMessage();
30 }
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek_4>main
Kurs ismi
Baska bir kurs ismi gir
fatma

Hello fatma
```


4-Kurucu Fonksiyonlar ile Nesnelere İlk Değer Atama

```
main.cpp
1 #include <iostream>
2 using namespace std;
3 class GradeBook{
4 public:
5     //courseName üye değişkenine string parametrelili kurucu ile ilk deger
6     explicit GradeBook(string name):courseName(name)//courseName üye değişkenine ilk değer atama
7     {
8         //boş gövde ve kurucu fonksiyon
9     }
10
11 void setCourseName(string name){
12     courseName=name;//ders ismini nesnede saklama
13 }
14 string getCourseName() const{
15     return courseName;//nesnenin courseName değişken değerini döndürür
16 }
17 void displayMessage()const{
18     cout<<"Hello " <<getCourseName ();
19     cout<<endl;
20 }
21 private:
22     string courseName; //GradeBook ders ismi
23 };
24
25 int main(){
26     // iki GradeBook nesnesi oluşturma
27     GradeBook myGradeBook1("myGradeBook1");
28     GradeBook myGradeBook2("myGradeBook2");
29     //GradeBook sınıfı nesnesi oluşturma
30     cout<<"gradebook1 created for course " <<myGradeBook1.getCourseName ()<<endl;
31     cout<<"gradebook2 created for course " <<myGradeBook2.getCourseName ()<<endl;
32 }
```

NOT

C++'da : operatörü, kurucu fonksiyon (constructor) çağrıldığında, sınıfın veri üyelerine doğrudan ilk değer atamak için kullanılır. Bu yönetime member initializer list (üye başlatma listesi) denir.

na AKALIN

C++'ta, : sembolü, bir sınıfın constructor'ı içinde üye değişkenlerin başlatılması için kullanılır.

explicit anahtar kelimesi, bir constructor'ın sadece açıkça çağrılmasını sağlar ve otomatik tür dönüşümünü engeller.

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_5>g++ -o main main.cpp
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_5>main
gradebook1 created for course myGradeBook1
gradebook2 created for course myGradeBook2
```

5-Yeniden kullanılması için Sınıfı Ayrı bir Dosyaya Yerleştirme

```
main.h x main.cpp x
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 class GradeBook{
5 public:
6     //courseName üye değişkenine string parametrelili kurucu ile ilk deger
7     explicit GradeBook(std::string name):courseName(name)//courseName üye değişkenine ilk deger atama
8     {
9         //boş gövde ve kurucu fonksiyon
10    }
11
12    void setCourseName(std::string name){
13        courseName=name;//ders ismini nesnede saklama
14    }
15    std::string getCourseName() const{
16        return courseName;//nesnenin courseName değişken deęerini döndürür
17    }
18    void displayMessage() const{
19        std::cout<<"Hello " <<getCourseName();
20        cout<<endl;
21    }
22 private:
23     std::string courseName; //GradeBook ders ismi
24 };
25
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek_6>g++ -c main.cpp -o main.o
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek_6>g++ main.o -o program
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek_6>program
```

```
gradebook1 created for course myGradeBook1
```

```
gradebook2 created for course myGradeBook2
```

```
main.h x main.cpp x
1 #include <iostream>
2 #include "main.h"
3 using namespace std;
4
5
6 int main(){
7     // iki GradeBook nesnesi oluşturma
8     GradeBook myGradeBook1("myGradeBook1");
9     GradeBook myGradeBook2("myGradeBook2");
10    //GradeBook sınıfı nesnesi oluşturma
11    cout<<"gradebook1 created for course " <<myGradeBook1.getCourseName() <<endl;
12    cout<<"gradebook2 created for course " <<myGradeBook2.getCourseName() <<endl;
13 }
```

```

main.cpp
1  #include <iostream>
2  #include "main.h"
3  using namespace std;
4      //string argumanlı kurucu ile courseName'e ilk deger atama
5  GradeBook::GradeBook(string name):courseName(name)//courseName ile üye değişkenine ilk değer atama
6  {
7      //boş gövde ve kurucu fonksiyon
8  }//GradeBook kurucu fonksiyon sonu
9
10 void GradeBook::setCourseName(string name){
11     courseName=name;//ders ismini nesnede saklama
12 }
13 string GradeBook::getCourseName() const{
14     return courseName;//nesnenin courseName değişken değerini döndürür
15 }
16 void GradeBook::displayMessage() const{
17     cout<<"Hello "<<getCourseName();
18     cout<<endl;
19 }
20
21 int main(){
22     GradeBook gradeBook1("Birinci");
23     GradeBook gradeBook2("İkinci");
24
25     cout<<"--Birinci--"<<gradeBook1.getCourseName()<<endl<<"--İkinci--"<<gradeBook2.getCourseName()<<endl;
26     cout<<endl;
27 }

```

```

main.cpp  main.h
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class GradeBook{
5  public:
6
7      explicit GradeBook(std::string);
8      void setCourseName(std::string);
9      std::string getCourseName() const;
10     void displayMessage() const;
11 private:
12     std::string courseName; //GradeBook ders ismi
13 };
14

```


:: kapsam çözünürlük operatörü olarak bilinir. Bu operatör **her bir üye fonksiyonu GradeBook sınıf tanımlamasına** bağlar. Yani sınıfın üye fonksiyonlarını ve üye değişkenlerini bildirir. Önceki her bir fonksiyon ismini **GradeBook:: sız bildirim yazma** durumunda GradeBook sınıfının bu fonksiyonları derleyici tarafından tanınmazlar. Derleyici bu fonksiyonları **main fonksiyonu gibi görür ve kayıp fonksiyonlar** olarak dikkate alır. Bu fonksiyonlar aynı zamanda global fonksiyonları olarak da adlandırılır. Bu tür fonksiyonlar bir nesne belirlemeksizin GradeBook sınıfının private datalarına erişemez veya sınıf fonksiyonlarını çağıramaz. Bundan dolayı derleyicinin **bu fonksiyonları derlemesinin imkanı yoktur.**

Bir sınıfın üye fonksiyonunun o sınıfın dışında tanımlandığı zaman sınıfı ismini ve kapsam çözünürlük operatörünü **::** fonksiyon adlarında sunmak hatalara sebep olur

Set Fonksiyonu ile Verinin Doğruluğunun Denetlenmesi

```
main.cpp x main.h x
1  #include <iostream>
2  #include "main.h"
3  using namespace std;
4  GradeBook::GradeBook(string name)
5  {
6      setCourseName(name);
7  } //GradeBook kurucu fonksiyon sonu
8
9  void GradeBook::setCourseName(string name){
10     if(name.size() <= 25)
11         courseName=name; //ders ismini nesnede saklama
12     if(name.size() > 25)
13         courseName=name.substr(0,25);
14 }
15
16 string GradeBook::getCourseName() const{
17     return courseName; //nesnenin courseName değişken değerini döndürür
18 }
19 void GradeBook::displayMessage() const{
20     cout<<"Hello " <<getCourseName();
21     cout<<endl;
22 }
23
24 int main(){
25     GradeBook gradeBook1("Birinci");
26     GradeBook gradeBook2("İkinci");
27
28     cout<<"--Birinci--" <<gradeBook1.getCourseName() <<endl <<"--İkinci--" <<gradeBook2.getCourseName() <<endl;
29     cout<<endl;
30
31     gradeBook1.setCourseName("set edilen birinci");
32     gradeBook2.setCourseName("set edilen ikinci");
33     cout<<"--Birinci--" <<gradeBook1.getCourseName() <<endl <<"--ikinci--" <<gradeBook2.getCourseName() <<endl;
34     cout<<endl;
35 }
```

```
main.cpp x main.h x
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class GradeBook{
5  public:
6
7      explicit GradeBook(std::string); //courseName'e ilk değer atayan kurucu
8      void setCourseName(std::string);
9      std::string getCourseName() const;
10     void displayMessage() const;
11 private:
12     std::string courseName; //GradeBook ders ismi
13 };
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_8>g++ -c main.cpp -o main.o
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_8>g++ main.o -o program
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_8>program
```

```
--Birinci--Birinci
```

```
--İkinci--İkinci
```

```
--Birinci--set edilen birinci
```

```
--ikinci--set edilen ikinci
```

KAYNAKÇA

- 1-Veri Yapıları ve Algoritmalar, 12. Baskı, Rifat Çölkesen
- 2-Nejat Yumuşak, fatih Adak Veri C++ ile veri yapıları, 3.baskı
- 3-C++ ile programlama, Dokuzuncu baskıdan çeviri, Palme Yayınları, Paul Deitel,Harvey Deitel, Ceviri Editörü Cemil Öz
- 4-Veri Yapıları, Hakan Kutucu
- 5-<https://chatgpt.com/>